

Fantasy Tournament Voting

Group 8 - DB_NOOBS: Angel Moreno, Krrithik Ezhilarasan, Gerbert Castro, Matthew Soto

CMPS 3420-01 • Fall 2025

October 27, 2025

Table of Contents

[Table of Contents](#)

[Changelog](#)

[Updates since Phase 1](#)

[Formatting of Phase 1 Document](#)

[Vote Classification](#)

[Title Page](#)

[User Primary Key Change](#)

[Updates since Phase 2](#)

[Removal of Matchup Entry Associative Entity](#)

[User Table Renamed](#)

[Constraint Adjustments](#)

[Added ENUM Constraints](#)

[Foreign Key and Relationship Validation](#)

[Updates since Phase 4](#)

[User Table Enhancement](#)

[Phase 1 – Conceptual Database Design](#)

[1.1 – Fact-Finding Techniques and Information Gathering](#)

[1.1.1 – Introduction to the Enterprise/Organization](#)

[1.1.2 – Motivation and Preliminary Research](#)

[1.1.3 – The Miniverse of Interest](#)

[1.1.4 – User Groups, Data Access, and Operations](#)

[1.1.5 – Summary of Entities and Relationships](#)

[1.2 – Conceptual Database Design](#)

[1.2.1 – Entity Descriptions](#)

[USER \(strong\)](#)

[TOURNAMENT \(strong\)](#)

[ENTRY \(strong\)](#)

[MATCHUP \(strong\)](#)

[MATCHUP ENTRY \(associative between MATCHUP and ENTRY\)](#)

[VOTE \(associative between USER and MATCHUP\)](#)

[1.2.2 – Relationship Descriptions](#)

[1.2.3 – ER Diagram](#)

[Phase 2 – Conversion from Conceptual to Relational Database](#)

[2.1 - Revisions to ER Diagram](#)

[2.2 - Conceptual to Logical Conversion Process](#)

[2.2.1 - Converting Entity Types to Relations](#)

[2.2.2 - Converting Relationship Types to Relations](#)

[2.3.1 Relation Schema](#)

[Semantic Constraints/Business Rules](#)

Phase 3 – Physical Implementation and Query Design

3.0 - Revisions to the Relational Model

3.1 - Database Implementation

3.1.1 Conversion Process

3.1.2 Schema

3.2 - Sample Queries

3.2.1 Q1:

3.2.2 Q2:

3.2.3 Q3:

3.2.4 Q4:

3.2.5 Q5:

3.2.6 Q6:

3.2.6 Q7:

3.2.8 Q8:

3.2.3 Q9:

3.2.10 Q10:

Phase 4 – Programming Logic for SQL

4.1 - Introduction

4.2 - Views

4.2.1 View 1: view_active_tournaments

4.2.2 View 2: view_tournament_entries

4.2.3 - View 3: active_matchups_with_votes

4.3 - Stored Procedures

4.3.1. register_user() - INSERT

4.3.2. delete_tournament() - DELETE

4.3.3 update_matchup_status() - UPDATE

4.3.4 get_tournament_stats() - procedure for statistical metrics

4.4 - Triggers

4.4.1 - Foreign Key Constraints

4.4.2 - Explicit Triggers

4.4.2.1 trg_entry_same_ins & trg_entry_same_upd

4.4.2.2 trg_matchup_min_votes_upd

4.4.2.3 trg_matchup_select_winner

4.4.2.3 Trg_entry_prevent_delete_active

Phase 5 – Database Programming and Graphical User Interface (GUI) Application Development

5.0 - GUI Development

5.0.1 Examples of CRUD Operations Supported in the GUI

5.0.2 Use of Database Programming in the GUI

5.0.3 Database Tables Used

5.1 - GUI Functionalities and User Groups

[5.1.1 - User Capabilities](#)

[5.1.2 - Screenshots and Walkthrough](#)

[5.2 - GUI Programming](#)

[5.2.1 - Database Layer Programming](#)

[5.2.1.1 Stored Procedures](#)

[5.2.1.2 Views](#)

[5.2.1.3 Triggers](#)

[5.2.1.4 New Programming Logic for Phase 5:](#)

[5.2.2 - Application Layer Programming](#)

[5.2.2.1 Database Connection \(config.php\)](#)

[5.2.2.2 Executing a Stored Procedure \(register_user\)](#)

[5.2.2.3 Deleting a Tournament via Stored Procedure \(delete_tournament.php\)](#)

[5.2.2.4 Using a View to Fetch Active Matchups \(admin.php\)](#)

[5.2.2.5 Executing Direct Queries \(tournaments.php\)](#)

[5.2.2.6 Calling get_tournament_stats \(statistics.php\)](#)

[5.2.2.7 Semantic Constraint Enforcement in PHP \(cast_vote.php\)](#)

[5.2.2.8 Ensuring Matchup Status Before Voting](#)

[5.2.2.9 Handling File Uploads for Profiles \(edit_profile.php\)](#)

[5.2.2.10 AJAX Constraint Enforcement \(check_displayname.php\)](#)

[5.2.2.11 Round Progression Polling \(check_round_change.php\)](#)

[5.2.2.12 Authentication Middleware](#)

[5.2.2.13 Server-Side API for Real-Time Check on Username](#)

[5.2.3 - Front-End Layer Programming](#)

[5.2.3.1 HTML Structure for Round Voting \(round8.php\)](#)

[5.2.3.2 CSS Styling for Brackets \(round8.css\)](#)

[5.2.3.3 AJAX: Checking Display Name Availability](#)

[5.2.3.4 AJAX Round Status Polling](#)

[5.2.3.5 Client-Side Form Validation \(register.php\)](#)

[5.2.3.6 HTML Form Example \(Profile Editing\)](#)

[5.2.3.7 Front-End Table Rendering \(statistics.php\)](#)

[5.2.3.8 Report Generation](#)

[5.2.3.9 Interactive UI Elements \(Tooltips\)](#)

Changelog

Updates since Phase 1

Formatting of Phase 1 Document

- Change: Converted list-style answers into paragraph-style responses.
- Reason: Addressed the professor's feedback that Phase 1 should use narrative paragraphs instead of lists.

Vote Classification

- Change: Reclassified Vote as a weak (associative) entity instead of a strong entity. Defined composite PK (user_id, matchup_id, entry_id) with cast_at as a non-key attribute.
- Reason: Vote cannot exist independently; it depends on User, MatchUp, and Entry, so it must be modeled as associative.

Title Page

- Change: Added all group members' full names and removed "Phase 1" specification from the title.
- Reason: To follow project formatting requirements for final submission.

User Primary Key Change

- Change: Changed the User relation so that 'user_id' is the only primary key. 'email' is now a normal attribute instead of being part of the key.
- Reason: Ensures a stable system-generated identifier. This avoids problems if users change their email addresses and maintains proper database design.

Updates since Phase 2

Removal of Matchup_Entry Associative Entity

- Change: Removed the Matchup_Entry table and added entryA_id and entryB_id as foreign keys in the Matchup table.
- Reason: Based on instructor feedback, this change simplifies the schema by representing both competing entries directly within Matchup, eliminating the need for an extra associative relation.

User Table Renamed

- Change: Renamed the table from 'User' to 'Users'.
- Reason: Avoided conflict with SQL reserved keywords and improved clarity in queries.

Constraint Adjustments

- Change: Added 'Check' constraints to prevent empty string values for key text attributes (displayname, email, password, and title).

- Reason: Ensures data integrity and enforces required field validation at the database level.

Added ENUM Constraints

- Change: Introduced ENUM types for role, tourneystatus, and status columns.
- Reason: Restricts possible values to a defined list ('admin', 'player', 'guest') for cleaner data and easier validation.

Foreign Key and Relationship Validation

- Change: Explicitly defined all foreign keys between tables (users, tournament, entry, matchup, vote) in the tables.sql file.
- Reason: Enforces referential integrity and ensures each record properly references its parent entity.

Updates since Phase 4

User Table Enhancement

- Change: Added profile_picture (VARCHAR) and bio (TEXT) columns to the users table.
- Reason: To support enhanced social features in the GUI application. This addition allows the "Tournament Bracket" page to visually identify the host who created that specific tournament with a profile picture and interactively display their bio via a tooltip on hover. It also enables users to customize their personal identity on the Profile Dashboard.

Phase 1 – Conceptual Database Design

1.1 – Fact-Finding Techniques and Information Gathering

1.1.1 – Introduction to the Enterprise/Organization

We are designing a lightweight web platform that lets people create single-elimination tournaments (for sports teams, e-sports, movies, memes, etc.) and crowd-vote the winners of each matchup. The platform serves organizers who set up tournaments and voters who cast votes during open rounds. Core activities include creating tournaments, adding entries, opening/closing rounds, voting on matchups, and advancing winners until a champion is selected.

1.1.2 – Motivation and Preliminary Research

We chose a voting-bracket tournament because it's fun, easy to demo, and feels like a simpler version of fantasy sports. Our intended audience is anyone who wants to vote on head-to-head matchups across different media (sports, movies, books, memes, etc.). We're keeping a classic bracket format and plan to show an image and short description for each entry in the UI. Scope is intentionally aligned with what we covered in class (accounts/roles, 1:M and M:N via associative entities, and basic aggregation for vote counts).

For preliminary research, we looked at common bracket patterns and UI examples online, discussed voting windows and preventing duplicate votes, and thought through simple tie-breaker options. We also considered features like betting, but did not go through with it since that's out of scope for this phase. Based on this, we're sticking to single-elimination with exactly two entries per matchup and one vote per user per matchup, and for ties, we have decided to go with allowing the lower seed to advance for simplicity.

1.1.3 – The Miniverse of Interest

Our business aims to include user accounts, tournaments, entries, matchups per round, voting windows, votes, and winner advancement in the database. The business will intentionally exclude features such as live sports or movie APIs, player statistics, reseeding, double-elimination formats, betting, scoring systems, and user comments.

Assumptions:

The platform will support only single-elimination tournaments. In this format, each matchup eliminates one entry, with winners advancing in a straight bracket progression until a single champion remains. To maintain bracket symmetry, each tournament must have an even number of entries. Ties will be resolved in favor of the lower-seeded entry, removing ambiguity in determining winners. The platform will not support reseeding; winners will

continue to advance according to the original bracket order, preserving the initial seeding logic and avoiding mid-tournament restructuring. Tournaments will be publicly available for browsing and participation, with basic role-based access control distinguishing between organizers and voting users.

Sample Data plan:

The sample data plan will include approximately 6 to 10 users to exercise user account management and voting scenarios. Between 2 and 3 active tournaments will be created to test tournament operations and related features. Each tournament will include around 8 to 16 entries, representing teams, movies, or other contest subjects. During voting rounds, the system will process approximately 8 to 30 votes per round, simulating typical user engagement levels while supporting aggregation and validation testing.

Risks/Out of Scope:

The platform will not integrate with external sports or live data APIs. Financial or payment processing systems will not be implemented, and no social or communication features, such as comments, chat, or forums, are planned. These topics are intentionally kept out of scope because they would create challenges with data consistency and add unnecessary complexity to the product's design.

Chosen Approach:

We narrowed it down to single-elimination brackets with an even number of entries for a predictable schema and simple advancement logic, considering the amount of time we have to work on the project throughout this class.

1.1.4 – User Groups, Data Access, and Operations

User Roles & Permissions:

Organizers are registered users with the organizer role. They have access to their tournaments, entries, matchups, and vote tallies. They can create or edit tournaments, add entries, open and close rounds, generate next-round matchups, view results, and announce winners. Voters are registered users with the default role, while guests may only browse. Voters can access public tournaments, open matchups, and results. Their operations include casting exactly one vote per open matchup and viewing results once rounds have closed.

1.1.5 – Summary of Entities and Relationships

- An Organizer can organize many Tournaments; each Tournament is organized by exactly one Organizer.
- A Tournament can have many Entries; each Entry belongs to exactly one Tournament.
- A Tournament can include many Matchups; each Matchup belongs to exactly one Tournament.

- A Matchup pairs exactly two Entries (slots A and B) via the associative entity MatchUp_Entry; an Entry can appear in many Matchups.
- A User can cast at most one Vote per Matchup; a Matchup can receive many Votes; each Vote is for exactly one Entry that participates in that Matchup.

Business rules:

- Each Matchup has exactly two distinct slots {A, B}.
- Vote(user, matchup) is unique; votes only count during the matchup's open window.
- A Vote.entry must be one of the two entries paired in that Matchup.
- Single-elimination only; an even number of entries.
- Tie-breaker: If votes are tied at close, the lower seed advances.
- No reseeding; winners advance in bracket order.
- Public tournaments; basic role-based access.

1.2 – Conceptual Database Design

1.2.1 – Entity Descriptions

USER (strong)

Attributes:

- user_id — Unique identifier; unique: yes; single-value; simple; stored.
- email — Login email; unique: no; single-value; simple; stored.
- password — Hashed password; unique: no; single-value; simple; stored.
- display_name — Public name; unique: no; single-value; simple; stored.
- role — Authorization role {user, organizer, admin}; unique: no; single-value; simple; stored.

TOURNAMENT (strong)

Attributes:

- tournament_id — Unique identifier; unique: yes; single-value; simple; stored.
- title — Tournament title; unique: no; single-value; simple; stored.
- description — Optional description; unique: no; single-value; simple; stored.
- status — {draft, open, closed, completed}; unique: no; single-value; simple; stored.
- start_at — Planned start datetime; unique: no; single-value; simple; stored.
- end_at — Planned end datetime; unique: no; single-value; simple; stored.
- user_id — Organizer user (FK to User); unique: no; single-value; simple; stored.

ENTRY (strong)

Attributes:

- entry_id — Unique identifier; unique: yes; single-value; simple; stored.
- name — Entry name (team, movie, etc.); unique: no; single-value; simple; stored.
- seed — Optional seed integer; unique: no; single-value; simple; stored.
- tournament_id — Owning tournament (FK to Tournament); unique: no; single-value; simple; stored.
- description — display simple entry description to users; unique: no; single-value; simple; stored.
- image_url — link to display image (TEXT/URL); unique: no; single-value; simple; stored.

MATCHUP (strong)

Attributes:

- matchup_id — Unique identifier; unique: yes; single-value; simple; stored.
- round_number — Round index starting at 1; unique: no; single-value; simple; stored.
- opens_at — Voting window open; unique: no; single-value; simple; stored.
- closes_at — Voting window close; unique: no; single-value; simple; stored.
- status — {scheduled, open, closed}; unique: no; single-value; simple; stored.11
- tournament_id — Owning tournament (FK to Tournament); unique: no; single-value; simple; stored.

MATCHUP_ENTRY (associative between MATCHUP and ENTRY)

Attributes:

- slot — Which side {A, B}; unique within one matchup; single-value; simple; stored.
- Identifying keys inherited from MATCHUP and ENTRY.
- Primary Key: (matchup_id, entry_id)
- Foreign Keys: matchup_id → MatchUp(matchup_id); entry_id → Entry(entry_id)
- Constraint: Unique(matchup_id, slot) (enforces one A and one B per matchup)

VOTE (associative between USER and MATCHUP)

Depends on: USER, MATCHUP

Attributes:

- cast_at — When the vote was placed; unique: no; single-value; simple; stored.
- Entry_id — Chosen entry (FK to Entry); unique: no; single-value; simple; stored.
- Primary Key: (user_id, matchup_id) one vote per user per matchup.
- Foreign Keys: user_id → User(user_id); matchup_id → MatchUp(matchup_id); entry_id → Entry(entry_id)

- Constraint: (matchup_id, entry_id) must exist in MatchUp_Entry(matchup_id, entry_id).

1.2.2 – Relationship Descriptions

Organizes (USER-TOURNAMENT)

- Description: A user creates and manages tournaments.
- Entities involved: USER, TOURNAMENT
- Cardinality: 1:M (one user, many tournaments).
- Participation: TOURNAMENT is mandatory/one; USER is optional/many.
- Attributes: none

For (MATCHUP-VOTE)

- Description: Each vote is tied to one specific matchup; a matchup can have many votes.
- Entities involved: MATCHUP, VOTE
- Cardinality: 1:M (one matchup, many votes).
- Participation: MATCHUP is optional/many; VOTE is mandatory/one.
- Attributes: none
- Constraints: PK(VOTE) = (user_id, matchup_id) enforces one vote per (user, matchup); FK (matchup_id, entry_id) → MATCHUP_ENTRY

ChosenBy (ENTRY-VOTE)

- Description: Each vote chooses exactly one entry; an entry may be chosen by many votes or none.
- Entities involved: ENTRY, VOTE
- Cardinality: 1:M (one entry, many votes).
- Participation: ENTRY is optional/many; VOTE is mandatory/one.
- Attributes: none
- Constraints: FK (matchup_id, entry_id) → MATCHUP_ENTRY (the chosen entry must be one of that matchup's two entries).

Has (TOURNAMENT-ENTRY)

- Description: A tournament contains entries (contestants) that participate in its bracket.
- Entities Involved: TOURNAMENT, ENTRY
- Cardinality: 1:M (TOURNAMENT: ENTRY)
- Participation: ENTRY is mandatory/one; TOURNAMENT is optional/many.
- Attributes: none

Includes (TOURNAMENT-MATCHUP)

- Description: A tournament is composed of matchups (bracketed contests).
- Entities Involved: TOURNAMENT, MATCHUP
- Cardinality: 1:M (TOURNAMENT: MATCHUP)
- Participation: MATCHUP is mandatory/one; TOURNAMENT is optional/many.
- Attributes: none

Casts (USER–MATCHUP–ENTRY via VOTE)

- Description: A user casts one vote in a specific matchup, selecting one of that matchup's two entries; stored in the associative entity 'Vote'.
- Entities Involved: User, MatchUp (chosen Entry referenced on Vote).
- Cardinality: M:N (User: MatchUp); uniqueness per (user, matchup).
- Participation: User is optional/many; MatchUp is optional/many.
- Relationship attribute: cast_at on VOTE.
- Constraints: PK(Vote) = (user_id, matchup_id); FK (matchup_id, entry_id) → MatchUp_Entry (vote must target one of that matchup's two entries); if tied at close, lower seed advances.

Includes_Pairing (MATCHUP–MATCHUP_ENTRY)

- Description: A matchup has pairing rows in MatchUp_Entry.
- Entities Involved: MatchUp, MatchUp_Entry.
- Cardinality: 1:M (MatchUp: MatchUp_Entry).
- Participation: MatchUp is mandatory/many; MatchUp_Entry is mandatory/one.
- Relationship attribute: none

Appears_In (ENTRY, MATCHUP_ENTRY)

- Description: An entry appears in pairing rows in MatchUp_Entry.
- Entities Involved: Entry, MatchUp_Entry.
- Cardinality: 1:M (Entry:MatchUp_Entry)
- Participation: Entry is optional/many; MatchUp_Entry is mandatory/one.
- Relationship attribute: none.

1.2.3 – ER Diagram

Notation: Crow's Foot with look-across labels. Model USER, TOURNAMENT, ENTRY, MATCHUP, MATCHUP_ENTRY (associative), and VOTE (associative). Draw relationships and cardinalities as described. Ensure that the USER includes a unique user_id and a password attribute.

Phase 2 – Conversion from Conceptual to Relational Database

2.1 - Revisions to ER Diagram

Our group opted to keep our ER Diagram as it was during our submission of Phase 1. We had no changes we wished to make as of this time, and figured that if anything came up, we would make changes accordingly. For now, the ER Diagram remains the same.

2.2 - Conceptual to Logical Conversion Process

2.2.1 - Converting Entity Types to Relations

Strong Entity Types:

Tournament (tournament_id PK, status, description, title, start_At, end_At)

User (user_id PK, role, email, password, display_name)

MatchUp (matchup_id PK, status, round_number, opens_at, closes_At)

Entry (entry_id PK, name, seed, description, image_URL)

Weak Entity Types:

MatchUp_Entry(matchup_id PK/FK, entry_id PK/FK, slot(a,b))

Vote (user_id PK/FK, matchup_id PK/FK, entry_id PK/FK, cast_At)

Both of these are associative entities, which are a type of weak entity that relies on strong entities for identification. They use composite primary keys made up of foreign keys referencing their parent relations. MatchUp_Entry references the primary keys of MatchUp and Entry, with 'slot' included as an attribute to indicate position within the matchup vote. Vote references the primary keys of User, MatchUp, and Entry, with 'cast_at' included to record when the vote was made.

Inheritance:

Our ER model does not use inheritance (specialization/generalization), so conversion strategies 8a–8d are not needed.

2.2.2 - Converting Relationship Types to Relations

Binary 1:M relationships

Organizes (User–Tournament):

Implemented by a 'user_id' foreign key in Tournament. Each Tournament must be organized by one Organizer, where their role as a normal user or an organizer will be determined by the role in the User relation. So only the Users with their roles assigned as organizers can be part of the Tournament relation and organize the tournaments.

Has (Tournament–Entry)

This is a 1:M relationship, where one Tournament has many Entries, but each Entry belongs to exactly one Tournament. To represent this, tournament_id is included as a foreign key in the Entry relation, referencing Tournament(tournament_id).

Includes (Tournament–Entry):

This is a 1:M relationship, where one Tournament includes many MatchUps, but each MatchUp belongs to exactly one Tournament. To represent this, tournament_id is added as a foreign key in the MatchUp relation, referencing Tournament(tournament_id).

Binary 1:M relationships within Associative Entities

Includes_Pairing (MatchUp–MatchUp_Entry):

Each MatchUp consists of exactly two MatchUp_Entries, but each MatchUp_Entry belongs to exactly one MatchUp. To show this, matchup_id is included in MatchUp_Entry as a foreign key referencing MatchUp(matchup_id). The composite key (matchup_id, entry_id) ensures the uniqueness of the pairing.

Appears_In (Entry–MatchUp_Entry):

Each Entry appears in many MatchUp_Entries, but each MatchUp_Entry corresponds to exactly one Entry. So, MatchUp_Entry has a foreign key entry_id referencing Entry(entry_id).

Vote (User–MatchUp–Entry):

Vote appears to be a ternary relationship among User, MatchUp, and Entry. In the relational model, it is represented as its own relation with foreign keys to each of the three parent relations: Vote(user_id PK/FK, matchup_id PK/FK, entry_id PK/FK, cast_at).

The point to note here is that the Vote relation references these three, but each Vote is linked to exactly one User, one MatchUp, and one Entry at a time, representing 1:M relationships rather than M: N or a ternary relation. This enforces that each user can vote once per entry per matchup, where cast_at is a non-key attribute indicating the vote time.

Multi-valued attributes:

No multi-valued attributes are present in the models, and each attribute is atomic.

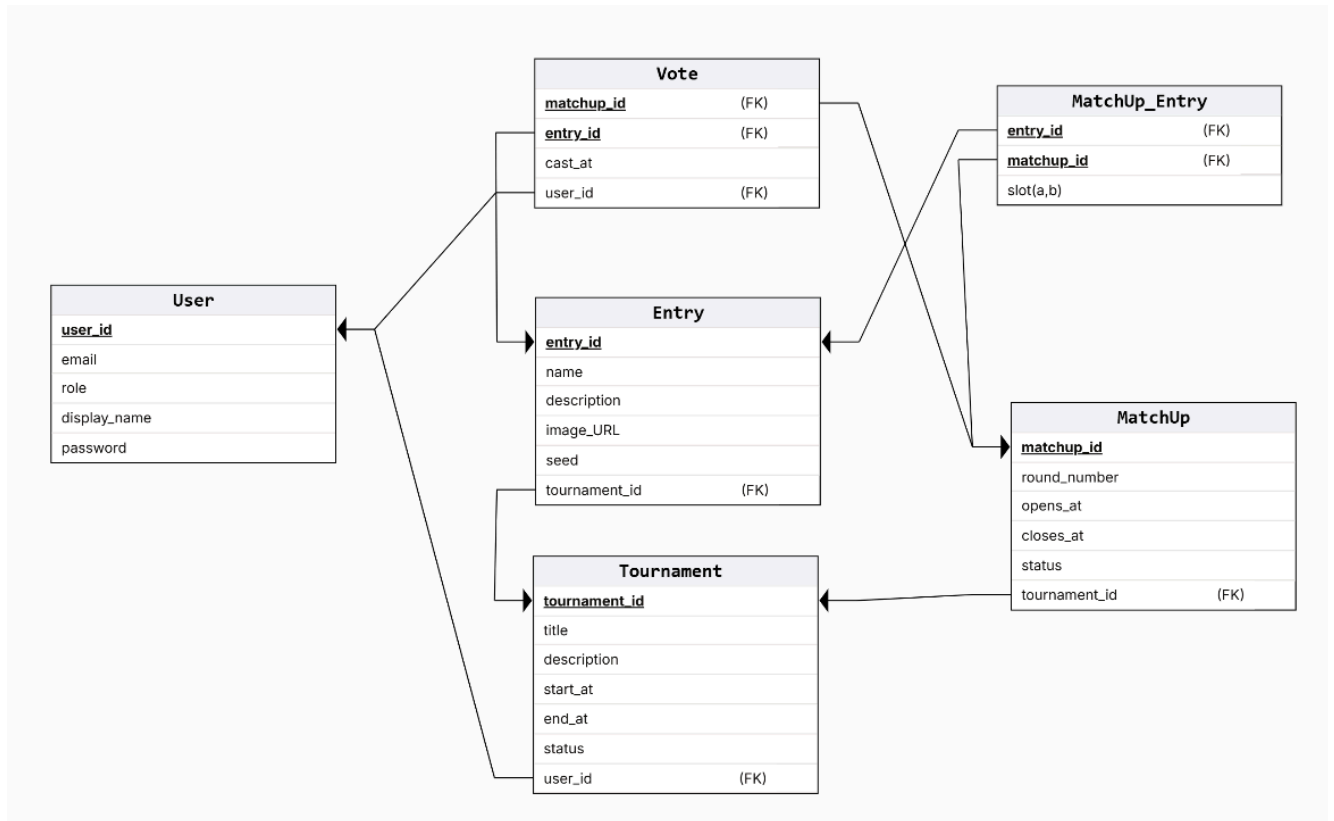
N-ary relationships:

Our model does not incorporate n-ary or ternary relationships. While the "Vote" relation might appear to be one, it is instead represented by multiple one-to-many (1:M)

relationships: User to Vote, MatchUp to Vote, and Entry to Vote. This structure is a direct result of our ER diagram's design, which reflects our product's business logic. Specifically, this allows each User, MatchUp, and Entry to be associated with multiple votes.

2.3 - Results of ER to Relational Conversion

2.3.1 Relation Schema



Semantic Constraints/Business Rules

Each matchup must have exactly two contestants, one in slot A and one in slot B. While the database can prevent duplicate slots, it cannot ensure that both slots are filled before publishing a matchup. From a business perspective, this means organizers must check that both slots are occupied before opening the matchup to the public.

Each user may only cast one vote per matchup, and that vote must be submitted while the matchup is open. Votes submitted before the open time or after the closing time are not valid. The application must enforce this rule because the database itself cannot check timestamps against the current time.

The entry that receives a vote must be one of the two contestants in that matchup. If admins later change the pairing of entries, any existing votes may no longer match the correct contestants. In this case, those votes must be intentionally resolved or voided by administrators.

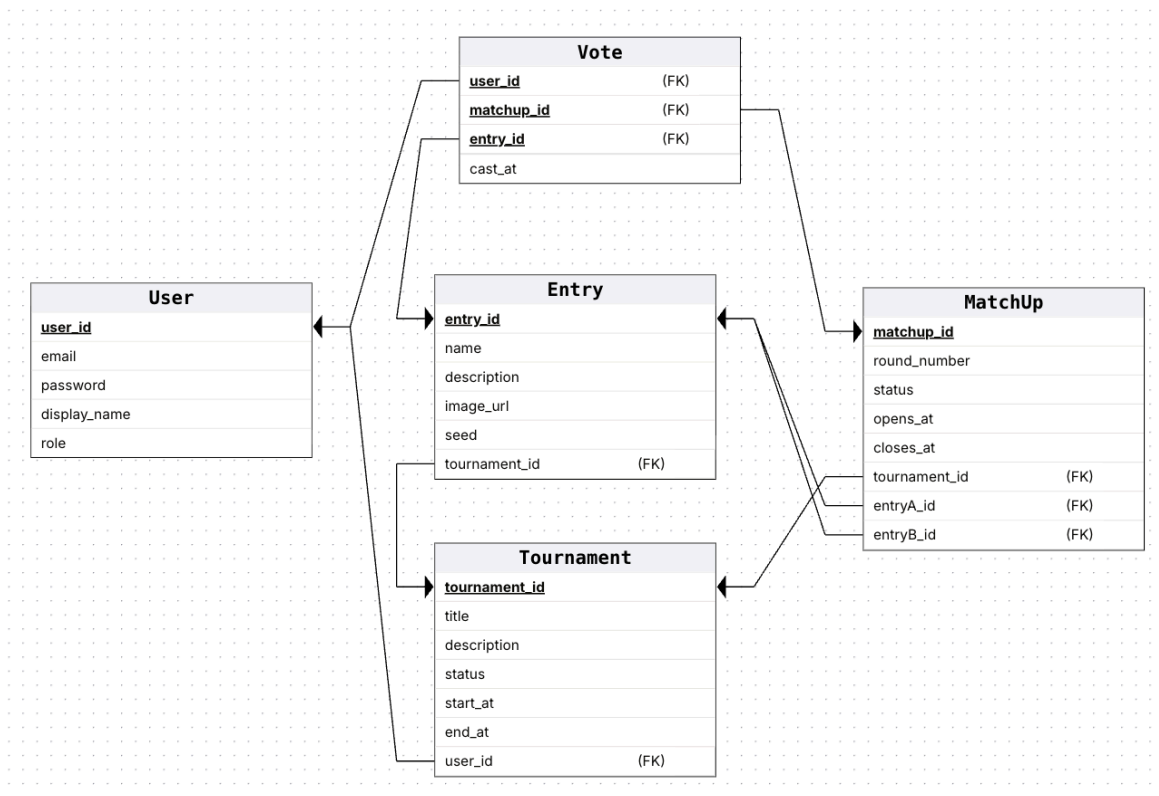
Tournaments must follow a single-elimination format and begin with an even number of entries (preferably a power of two). The database cannot enforce this requirement, so it is the responsibility of organizers to ensure proper tournament setup. In the event of a tie after voting closes, the entry with the lower seed automatically advances, which is a business rule outside the scope of the relational model.

Finally, only users with the role of organizer may create or edit tournaments. General users are limited to browsing tournaments and voting in matchups when they are open.

Phase 3 – Physical Implementation and Query Design

3.0 - Revisions to the Relational Model

Since Phase 2, we revised our relational model to simplify the representation of matchups. At first, we used an associative entity 'Matchup_Entry' to link entries to matchups. Following the instructor's feedback, we decided to eliminate that table and instead added two foreign keys (entryA_id and entryB_id) directly in the Matchup table to represent the two competing entries. This change made queries simpler to write. All other tables remained consistent with Phase 2, but we renamed user -> users to avoid conflicts with reserved keywords and ensure cleaner references in foreign keys.



3.1 - Database Implementation

3.1.1 Conversion Process

An issue we ran into was holding our multivalued attributes. That is when we landed on the ENUM data type, which is recommended for storing strings and is accessed by their index, which works for columns like 'role' in the table 'users'. We did not change the primary key of any relation when implementing them as tables. Our group did not encounter many difficulties translating the relational schema to SQL.

When unsure of implementation, we reference the MariaDB documentation to finalize decisions for issues.

The kinds of data we inserted into our tables may consist of date, time, emails, IDs, and titles, to name a few. Data collected was generated by ChatGPT and then used to populate our tables. Our tables hold mock data and do not represent anything in the real world, though it is similar in appearance to real data.

3.1.2 Schema

Our database schema is implemented through a set of three SQL scripts located in our shared Artemis workspace under the 'fantasytourneydb' directory. These scripts allow us to drop, recreate, and repopulate the database at any time for testing or demonstration. Each file serves a distinct purpose as described below.

Tables.sql: This file contains all CREATE TABLE statements for the database, including primary keys, foreign keys, and integrity constraints. Before creating new tables, existing ones are dropped to prevent duplication.

CODE:

```
select "Creating the Fantasy Tourney DB tables. ";

select "Creating tables with no foreign keys first: User";

drop table if exists vote;
drop table if exists matchup;
drop table if exists entry;
drop table if exists tournament;
drop table if exists users;

-- create User table
create table users (
    user_id int primary key auto_increment,
    email varchar(255) unique not null check(email <> ''),
    role ENUM('admin', 'player', 'guest') not null,
    displayname varchar(100) not null check(displayname <> ''),
    password varchar(255) not null check(password <> '')
);

-- create Tournament table
create table tournament (
    tournament_id int primary key auto_increment,
    title varchar (255) unique not null check(title <> ''),
    description varchar(255) not null check(description <> ''),
    start_at date not null,
    end_at date default null,
    tourneystatus ENUM('Upcoming', 'Active', 'Completed') not null,
    user_id int not null,
    foreign key (user_id) references users(user_id)
);
```

```

-- create Entry table
create table entry (
    entry_id int primary key auto_increment,
    name varchar(100) not null check(name <> ''),
    description varchar(255) default null,
    image_URL varchar(255) default null,
    seed int default null,
    tournament_id int not null,
    foreign key (tournament_id) references tournament(tournament_id)
);

-- create MatchUp table
create table matchup (
    matchup_id int primary key auto_increment,
    round_number int not null,
    status enum('Upcoming', 'Active', 'Completed') not null,
    opens_at datetime not null,
    closes_at datetime not null,
    tournament_id int not null,
    entryA_id int not null,
    entryB_id int not null,
    foreign key (tournament_id) references tournament(tournament_id),
    foreign key (entryA_id) references entry(entry_id),
    foreign key (entryB_id) references entry(entry_id),
    check (entryA_id <> entryB_id)
);

-- create Vote table;
create table vote (
    user_id int not null,
    matchup_id int not null,
    entry_id int not null,
    cast_at datetime default current_timestamp,
    primary key (user_id, matchup_id),
    foreign key (user_id) references users(user_id),
    foreign key (matchup_id) references matchup(matchup_id),
    foreign key (entry_id) references entry(entry_id)
);

```

Data.sql: This file populates the database with mock data for testing and demonstration. It includes delete statements at the top so that the dataset can be safely reloaded without causing duplicate entry errors.

CODE:

```

-- Fantasy Tournament Sample Data
-- This file will insert test data for tables

-- helps us delete and reload data safely --

```

```

-----
set foreign_key_checks = 0;      -- turns off FK enforcement
delete from vote;
delete from matchup;
delete from entry;
delete from tournament;
delete from users;
set foreign_key_checks = 1;      -- turns on FK enforcement
-----

-- Users --
-----
insert into users (email, role, displayname, password)
values
('jsoto24@csub.edu', 'admin', 'slug', 'dbnoob1'),
('krithickdagod@csub.edu', 'player', 'krrxthxk', 'dbnoob2'),
('angelBchillen@csub.edu', 'guest', 'angelB', 'dbnoob3'),
('gerbertdadude@csub.edu', 'player', 'AZ', 'dbnoob4');

-- Tournaments --
-----
insert into tournament (title, description, start_at, end_at, tourneystatus,
user_id)
values
('Autumn Showdown', 'A seasonal battle between the best entries.',
'2025-09-01', '2025-09-15', 'Completed', 1),
('Winter Clash', 'Frosty fights for the winter crown.', '2025-12-01',
'2025-12-15', 'Upcoming', 1);

-- Entries --
-----
insert into entry (name, description, image_URL, seed, tournament_id)
values
('Crimson Blades', 'A fierce red-themed team.',
'https://example.com/img/crimson.png', 1, 1),
('Azure Storm', 'The team with unmatched agility.',
'https://example.com/img/azure.png', 2, 1),
('Golden Hawks', 'Rising stars of the tournament.',
'https://example.com/img/hawks.png', 3, 2),
('Emerald Titans', 'Green giants of the Winter Clash.',
'https://example.com/img/titans.png', 4, 2);

-- MatchUps --
-----
insert into matchup (round_number, status, opens_at, closes_at,
tournament_id, entryA_id, entryB_id)
values
(1, 'Completed', '2025-09-01 10:00:00', '2025-09-01 22:00:00', 1, 1, 2),
(1, 'Upcoming', '2025-12-01 10:00:00', '2025-12-01 22:00:00', 2, 3, 4);

-- Votes --
-----
insert into vote (user_id, matchup_id, entry_id)
values
(2, 1, 1), -- Krrxthxk voted for Crimson Blades in Autumn Showdown

```

```
(3, 1, 2), -- angelB voted for Azure Storm
(4, 2, 3); -- AZ voted for Golden Hawks
```

Fantasy_tournament.sql: This is the master rebuild script. It drops all existing tables, then sources tables.sql to recreate them and data.sql to insert data. Running this script completely resets the database in a single command.

Master rebuild command: **source fantasytourneydb/fantasy_tournament.sql;**

CODE:

```
-- fantasy_tournament.sql
-- Master rebuild script for the Fantasy Tournament Database

-- Helps us drop tables freely --
-----
SET foreign_key_checks = 0;

-- Drop tables if they exist (in reverse dependency order)
DROP TABLE IF EXISTS vote;
DROP TABLE IF EXISTS matchup;
DROP TABLE IF EXISTS entry;
DROP TABLE IF EXISTS tournament;
DROP TABLE IF EXISTS users;

SET foreign_key_checks = 1;

-- Recreate the schema
SOURCE fantasytourneydb/tables.sql;

-- Optionally insert mock data
SOURCE fantasytourneydb/data.sql;
```

3.2 - Sample Queries

3.2.1 Q1:

PE: Find tournaments with 'Winter Clash' as the title.

RA: $T \leftarrow \sigma_{\text{title} = \text{"basketball"}}(\text{Tournament})$

SQL:

```
SELECT *
FROM tournament
```

```
WHERE title = 'Winter Clash';
```

3.2.2 Q2:

PE: Find users who are tournament organizers.

RA: $U \leftarrow \sigma_{\text{role} = \text{"organizer"}}(\text{User})$

RESULT $\leftarrow \pi_{\text{display_name}}(U)$

SQL:

```
SELECT displayname
FROM users
WHERE role = 'admin';
```

3.2.3 Q3:

PE: Find the names of entries that start at 11 AM.

RA: $T \leftarrow \sigma_{\text{start_at} = \text{"11 AM"}}(\text{Tournament})$

$E \leftarrow T \bowtie_{T.\text{tournament_id} = \text{Entry.tournament_id}} \text{Entry}$

RESULT $\leftarrow \pi_{\text{name}}(E)$

SQL:

```
SELECT e.name
FROM tournament t
JOIN entry e ON t.tournament_id = e.tournament_id
WHERE t.start_at = '11:00:00';
```

3.2.4 Q4:

PE: Find the emails of users who cast votes on the matchup with ID 214324.

RA: $V \leftarrow \sigma_{\text{matchup_id} = \text{"214324"}}(\text{Vote})$

$U \leftarrow V \bowtie_{V.\text{user_id} = \text{User.user_id}} \text{User}$

RESULT $\leftarrow \pi_{\text{email}}(U)$

SQL:

```
SELECT u.email
FROM vote v
JOIN users u ON v.user_id = u.user_id
WHERE v.matchup_id = 214324;
```

3.2.5 Q5:

PE: Show all votes and who voted for what

RA: $\pi_{\text{displayname, matchup_id, name, cast_at}}(((\text{Vote} \bowtie_{\text{Vote.user_id}=\text{Users.user_id}} \text{Users}) \bowtie_{\text{Vote.matchup_id}=\text{Matchup.matchup_id}} \text{Matchup}) \bowtie_{\text{Vote.entry_id}=\text{Entry.entry_id}} \text{Entry}))$

SQL:

```
SELECT u.displayname, m.matchup_id, e.name AS voted_for, v.cast_at
FROM vote v
JOIN users u ON v.user_id = u.user_id
JOIN matchup m ON v.matchup_id = m.matchup_id
JOIN entry e ON v.entry_id = e.entry_id;
```

3.2.6 Q6:

PE: Show each matchup and the two entries competing

RA: $\pi_{\text{matchup_id, E1.name, E2.name}}((\text{Matchup} \bowtie_{\text{Matchup.entryA_id}=\text{E1.entry_id}} \rho_{\text{E1}}(\text{Entry})) \bowtie_{\text{Matchup.entryB_id}=\text{E2.entry_id}} \rho_{\text{E2}}(\text{Entry}))$

SQL:

```
SELECT m.matchup_id, e1.name AS entry_A, e2.name AS entry_B
FROM matchup m
JOIN entry e1 ON m.entryA_id = e1.entry_id
JOIN entry e2 ON m.entryB_id = e2.entry_id;
```

3.2.6 Q7:

PE: List all tournaments and their creators

RA: $\pi_{\text{title, displayname}}(\text{Tournament} \bowtie_{\text{Tournament.user_id}=\text{Users.user_id}} \text{Users})$

SQL:

```
SELECT t.title, u.displayname AS created_by
FROM tournament t
JOIN users u ON t.user_id = u.user_id;
```

3.2.8 Q8:

PE: Find the tournament that has the highest average number of votes per matchup

RA: $T1 \leftarrow \gamma_{\text{matchup_id}; \text{count}(\text{user_id}) \rightarrow \text{vote_count}}(\text{Vote})$

$T2 \leftarrow T1 \bowtie_{T1.matchup_id = \text{Matchup.matchup_id}} \text{Matchup}$

$T_{\text{avg}} \leftarrow \gamma_{\text{tournament_id}; \text{avg}(\text{vote_count}) \rightarrow \text{avg_votes}}(T2)$

$T_{\text{max}} \leftarrow \sigma_{\text{avg_votes} = \text{MAX}(\text{avg_votes})}(T_{\text{avg}})$

$R \leftarrow T_{\text{max}} \bowtie_{T_{\text{max.tournament_id}} = \text{Tournament.tournament_id}} \text{Tournament}$

$R \leftarrow \pi_{\text{title}}(R)$

SQL:

```
WITH votes_per_matchup AS (
```

```

        SELECT matchup_id, COUNT(user_id) AS vote_count
        FROM vote
        GROUP BY matchup_id
    ),
    avg_votes_per_tournament AS (
        SELECT m.tournament_id, AVG(v.vote_count) AS avg_votes
        FROM votes_per_matchup v
        JOIN matchup m ON v.matchup_id = m.matchup_id
        GROUP BY m.tournament_id
    )
    SELECT t.title
    FROM avg_votes_per_tournament a
    JOIN tournament t ON a.tournament_id = t.tournament_id
    WHERE a.avg_votes = (
        SELECT MAX(avg_votes) FROM avg_votes_per_tournament
    );

```

3.2.3 Q9:

PE: Find users who have never cast a vote

RA: $T1 \leftarrow \pi_{\text{user_id}}(\text{User})$

$T2 \leftarrow \pi_{\text{user_id}}(\text{Vote})$

$R \leftarrow T1 - T2$

SQL:

```

SELECT u.user_id, u.email
FROM users u
WHERE u.user_id NOT IN (
    SELECT v.user_id FROM vote v
);

```

3.2.10 Q10:

PE: To identify users who have not voted in every open matchup in a tournament

RA: $\text{OpenMatchups} \leftarrow \pi_{\text{matchup_id}}(\sigma_{\text{status} = \text{"open"}}(\text{Matchup}))$

$\text{UserVotes} \leftarrow \pi_{\text{user_id}, \text{matchup_id}}(\sigma_{\text{status} = \text{"open"}}(\text{Vote} \bowtie \text{Matchup}))$

$\text{UsersWhoVotedForAll} \leftarrow \text{UserVotes} \div \text{OpenMatchups}$

$\text{AllUsers} \leftarrow \pi_{\text{user_id}}(\text{User})$

$\text{UsersPendingVotes} \leftarrow \text{AllUsers} - \text{UsersWhoVotedForAll}$

SQL:

```

-- Users who have not voted in every open matchup
WITH open_matchups AS (
    SELECT matchup_id
    FROM matchup
    WHERE status = 'open'
),
user_votes AS (

```

```
        SELECT v.user_id, v.matchup_id
        FROM vote v
        JOIN matchup m ON v.matchup_id = m.matchup_id
        WHERE m.status = 'open'
    ),
    users_voted_all AS (
        SELECT uv.user_id
        FROM user_votes uv
        GROUP BY uv.user_id
        HAVING COUNT(DISTINCT uv.matchup_id) = (SELECT COUNT(*) FROM
open_matchups)
    )
    SELECT u.user_id, u.email
    FROM users u
    WHERE u.user_id NOT IN (SELECT user_id FROM users_voted_all);
```

Phase 4 – Programming Logic for SQL

4.1 - Introduction

SQL programming logic is different from other languages because it relies on an existing structure. This structure is also known as a relational database. Queries cannot be executed without knowledge of the existing tables and how they are connected through primary and foreign keys. SQL statements will work in appropriate environments however the same queries will not run everywhere.

- View (fetches data from tables and is presented in a customized format)
 - A. Giving users an abstracted layer allows them to easily make an account and provide tournaments for the community to participate in
 - B. Allows control of data access points which is crucial for security reasons
- Triggers (SQL statements that are invoked when an event occurs on a table.)
 - A. Save results of the current match when the voting window is closes
- Stored Procedures (Recurring SQL query)
 - A. INSERT Tournament
 - B. INSERT Account
 - C. UPDATE Tournament
 - D. UPDATE Account

4.2 - Views

This section presents three SQL views created for our Fantasy Tournament application. Each view includes the SQL command used to create it, a description of why the view is useful, and an example query using the view with its returned results. These views simplify repeated queries and support features that will be used in the future.

4.2.1 View 1: *view_active_tournaments*

The application frequently needs to display a list of currently active tournaments. Instead of rewriting a long SELECT statement every time, this view merges the logic. It improves readability, reduces repeated SQL code, and supports fast filtering during tournament browsing.

```
-- View 1
CREATE VIEW view_active_tournaments AS
SELECT
    tournament_id,
    title,
    description,
    start_at,
    end_at,
    tourneystatus,
    user_id
FROM tournament
WHERE tourneystatus = 'active';
```

```
-- Sample Query

SELECT title, start_at, end_at
FROM view_active_tournaments
ORDER BY start_at ASC;

-- Shows active tournaments in order of their start date.
-- Results:
```

title	start_at	end_at
Best 90s Movie	2024-11-01	NULL
Greatest Video Game	2024-11-10	NULL

4.2.2 View 2: *view_tournament_entries*

Entries appear throughout the application—in matchups, brackets, leaderboards, and tournament details. Since the system frequently needs to list entries for a specific tournament, this view puts that logic into a single reusable component. It simplifies queries for displaying tournament contents in the GUI and reduces repeated JOIN statements.

```
DROP VIEW IF EXISTS view_tournament_entries;

CREATE VIEW view_tournament_entries AS
SELECT
    t.tournament_id,
    t.title AS tournament_title,
    e.entry_id,
    e.name AS entry_name,
    e.image_url
FROM tournament t
```

```
JOIN entry e ON t.tournament_id = e.tournament_id
ORDER BY t.tournament_id, e.entry_id;
```

```
-- Sample Query
```

```
SELECT tournament_title, entry_id, entry_name
FROM view_tournament_entries
WHERE tournament_id = 1;
```

```
-- Retrieves all entries belonging to tournament #1, including the tournament
-- title, each entry's ID and each entry's name.
-- Results:
```

tournament_title	entry_id	entry_name
Best Pizza Toppings 2024	1	Pepperoni
Best Pizza Toppings 2024	2	Mushrooms
Best Pizza Toppings 2024	3	Sausage
Best Pizza Toppings 2024	4	Green Peppers
Best Pizza Toppings 2024	5	Onions
Best Pizza Toppings 2024	6	Black Olives
Best Pizza Toppings 2024	7	Extra Cheese
Best Pizza Toppings 2024	8	Bacon

4.2.3 - View 3: active_matchups_with_votes

Displays all the active matchups with real-time vote counts for each competitor(entry). This view can be used to view live voting result of active tournaments on the frontend , maybe is the admin want to see live progress

```

DROP VIEW IF EXISTS active_matchups_with_votes;

CREATE VIEW active_matchups_with_votes AS
SELECT
    m.matchup_id,
    t.tournament_id,
    t.title AS tournament_title,
    m.round_number,
    m.status,
    m.opens_at,
    m.closes_at,
    e1.entry_id AS entryA_id,
    e1.name AS entryA_name,
    e1.image_URL AS entryA_image,
    e2.entry_id AS entryB_id,
    e2.name AS entryB_name,
    e2.image_URL AS entryB_image,
    COUNT(DISTINCT CASE WHEN v.entry_id = m.entryA_id THEN v.user_id END) AS
entryA_votes,
    COUNT(DISTINCT CASE WHEN v.entry_id = m.entryB_id THEN v.user_id END) AS
entryB_votes,
    COUNT(DISTINCT v.user_id) AS total_votes
FROM matchup m
JOIN tournament t ON m.tournament_id = t.tournament_id
JOIN entry e1 ON m.entryA_id = e1.entry_id
JOIN entry e2 ON m.entryB_id = e2.entry_id
LEFT JOIN vote v ON m.matchup_id = v.matchup_id
WHERE m.status = 'Active'
GROUP BY
    m.matchup_id, t.tournament_id, t.title, m.round_number,
    m.status, m.opens_at, m.closes_at,
    e1.entry_id, e1.name, e1.image_URL,
    e2.entry_id, e2.name, e2.image_URL;

```

```

-- A useful sample query on this view ->

SELECT tournament_title, round_number, entryA_name, entryA_votes, entryB_name,
    entryB_votes, closes_at
FROM active_matchups_with_votes
ORDER BY closes_at ASC;

```

4.3 - Stored Procedures

4.3.1. *register_user()* - INSERT

This procedure handles new user registration with validation. Ensures email uniqueness and proper data formatting before creating an account. Also ensures data integrity by checking for empty fields, validating role values, and preventing the insertion of duplicate email

registrations. This handles the entire workflow of a new user creation in a single atomic operation

This procedure will be called from the signup page on the frontend. When the user submits the registration form, the GUI application will execute this procedure with the user-provided credentials. The procedure performs all the server-side validation, thus preventing any invalid data from entering the database and providing necessary error messages for the user interface to display on the website.

```
DELIMITER //
```

```
CREATE PROCEDURE register_user(  
    p_email VARCHAR (255),  
    p_password VARCHAR(255),  
    p_displayname VARCHAR (100),  
    p_role VARCHAR (20)  
)  
  
BEGIN  
    declare email_exists int default 0;  
  
    if p_email = "" or p_email is null then  
        select "ERROR: Email is required" as message;  
  
    elseif p_password = "" or p_password is null then  
        select "ERROR: Password is required" as message;  
  
    elseif p_displayname = "" or p_displayname is null then  
        select "ERROR: Displayname is required" as message;  
  
    elseif p_role not in ("admin", "player", "guest") then  
        select "ERROR: Invalid Role" as message;  
  
    else  
        -- check if email already exists  
        select count(*) into email_exists  
        from users  
        where email = p_email;  
  
        if email_exists > 0 then  
            select "ERROR: Email already exists" as message;  
        else  
            insert into users (email, password, displayname, role)  
            values (p_email, p_password, p_displayname, p_role);  
  
            -- returning successfully inserted user info row  
            select "User registered successfully!" as message;  
  
            select * from users  
            where email = p_email;  
        end if;  
    end if;  
END
```

```
END //

DELIMITER ;
```

Command to execute procedure:

```
CALL register_user('newuser@gmail.com', 'securepwd123', 'NewUser', 'Player');
```

Result:

```
MariaDB [fantasytourney]> CALL register_user('newuser@gmail.com', 'securepwd123', 'NewUser', 'Player');
+-----+
| message |
+-----+
| User registered successfully! |
+-----+
1 row in set (0.002 sec)

+-----+-----+-----+-----+-----+
| user_id | email | role | displayname | password |
+-----+-----+-----+-----+-----+
| 21 | newuser@gmail.com | player | NewUser | securepwd123 |
+-----+-----+-----+-----+-----+
1 row in set (0.002 sec)

Query OK, 2 rows affected (0.003 sec)
```

```
MariaDB [fantasytourney]> select * from users where email = 'newuser@gmail.com';
+-----+-----+-----+-----+-----+
| user_id | email | role | displayname | password |
+-----+-----+-----+-----+-----+
| 21 | newuser@gmail.com | player | NewUser | securepwd123 |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [fantasytourney]>
```

4.3.2. delete_tournament() - DELETE

Safely deletes old tournaments if admins want to and also all of the associated data (entries, matchups, votes) relating to that specific tournament in a single transaction. Validation prevents deletion of active tournaments.

This procedure was created in order to safely remove the tournaments and all associated data (entries, matchups, votes) in the correct order with respect to foreign key relationships. It includes business logic to prevent deletion of active tournaments, protecting ongoing tournaments where players are competing from accidental removal.

This procedure is planned to be used by the application's administrators to clean up old/completed tournaments if they feel that they are no longer needed. Thus, removing archived tournaments takes up waste space and all related data, maintaining foreign key referential integrities at the same time, preventing deletion of tournaments that are still in progress

```

DELIMITER //

CREATE PROCEDURE delete_tournament(
    p_tournament_id INT
)
BEGIN
    DECLARE tournament_exists int default 0;
    DECLARE tournament_status varchar(30);
    DECLARE active_matchup_count int default 0;

    -- check if tournament exists
    select count(*) into tournament_exists
    from tournament
    where tournament_id = p_tournament_id;

    if tournament_exists = 0 then
        select "Error: Tournament does not exist/ not found" as message;
    else
        -- get the status of tournament
        select tourneystatus into tournament_status
        from tournament
        where tournament_id = p_tournament_id;

        -- checking if tournament as active matchups
        select count(*) into active_matchup_count
        from matchup
        where tournament_id = p_tournament_id
        and status = "Active";

        if tournament_status = "Active" then
            select "Error: Cannot delete an active tournament" as message;
        elseif active_matchup_count > 0 then
            select concat("Error: Cannot delete tournament with",
                active_matchup_count, "active matchup(s). Cancel them first")
as message;
        else
            -- everything is okay to delete right now
            delete from vote
            where matchup_id in (
                select matchup_id from matchup where tournament_id =
p_tournament_id
            );

            delete from matchup where tournament_id = p_tournament_id;

            delete from entry where tournament_id = p_tournament_id;

            delete from tournament where tournament_id = p_tournament_id;

            select concat("Tournament Id ", p_tournament_id, " and related
data are deleted") as message;
        end if;
    end if;
END
//

```

```
END //
DELIMITER ;
```

Command to execute procedure:

```
CALL delete_tournament(1);
```

```
MariaDB [fantasytourney]> select t.tournament_id, t.title, t.tourneystatus,
-> count(distinct e.entry_id) as entries,
-> count(distinct m.matchup_id) as matchups,
-> count(v.user_id) as votes
-> from tournament t
-> left join entry e on t.tournament_id = e.tournament_id
-> left join matchup m on t.tournament_id = m.tournament_id
-> left join vote v on m.matchup_id = v.matchup_id
-> where t.tournament_id = 1
-> group by t.tournament_id, t.title, t.tourneystatus;
+-----+-----+-----+-----+-----+-----+
| tournament_id | title                | tourneystatus | entries | matchups | votes |
+-----+-----+-----+-----+-----+-----+
| 1 | Best Pizza Toppings 2024 | Completed     | 9       | 7        | 324   |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.003 sec)
```

```
MariaDB [fantasytourney]> select t.tournament_id, t.title, t.tourneystatus,
-> count(distinct e.entry_id) as entries,
-> count(distinct m.matchup_id) as matchups,
-> count(v.user_id) as votes
-> from tournament t
-> left join entry e on t.tournament_id = e.tournament_id
-> left join matchup m on t.tournament_id = m.tournament_id
-> left join vote v on m.matchup_id = v.matchup_id
-> where t.tournament_id = 1
-> group by t.tournament_id, t.title, t.tourneystatus;
+-----+-----+-----+-----+-----+-----+
| tournament_id | title                | tourneystatus | entries | matchups | votes |
+-----+-----+-----+-----+-----+-----+
| 1 | Best Pizza Toppings 2024 | Completed     | 9       | 7        | 324   |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.003 sec)
```

```
MariaDB [fantasytourney]> CALL delete_tournament(1);
+-----+-----+
| message                |
+-----+-----+
| Tournament Id 1 and related data are deleted |
+-----+-----+
1 row in set (0.008 sec)

Query OK, 56 rows affected (0.008 sec)
```

```
MariaDB [fantasytourney]> select * from tournament where tournament_id = 1;
Empty set (0.000 sec)
```

```
MariaDB [fantasytourney]> select * from entry where tournament_id = 1;
Empty set (0.000 sec)
```

```
MariaDB [fantasytourney]> select * from matchup where tournament_id = 1;
Empty set (0.001 sec)
```

4.3.3 update_matchup_status() - UPDATE

The procedure allows control of the entire phase of the matchups, ie, right from creation to completion of each matchup by updating their status based on the primary key (matchup_id). Matchup progresses through 3 phases: 'Upcoming', 'Active', 'Completed'. This procedure provides controlled state changes of matchup status with validation to prevent invalid status values and redundant updates.

Administrators of the application use this procedure to manage the tournament flow. When a round is ready to begin, the admin sets matchups to 'Active' to open voting. After voting closes, the admin can mark matchups as 'Completed'. This can be integrated with one of the admin dashboards, which serves as a control over how the matchup rounds are paced throughout the tournament. In the upcoming GUI development (phase 5), this could be automated based on the opens_at and closes_at timestamps, as well as serve as manual control for the admin, allowing flexibility to alter changes to the tournament as the admin desires.

```
DELIMITER //
CREATE PROCEDURE update_matchup_status(
    p_matchup_id INT,
    p_new_status varchar(30)
)
BEGIN
    DECLARE matchup_exists INT DEFAULT 0;
    DECLARE old_status VARCHAR (30);
    DECLARE tournament_title VARCHAR (255);

    -- check if matchup exist
    select count(*) into matchup_exists
    from matchup
    where matchup_id = p_matchup_id;

    if matchup_exists = 0 then
        select "Error: Matchup not found" as message;
    elseif p_new_status NOT IN ('Upcoming', 'Active', 'Completed') THEN
        -- Validation check for allowed status
        SELECT CONCAT('Error: Invalid status ', p_new_status,
            '". Allowed values: Upcoming, Active, Completed.') AS
message;
    else

        -- get current status and tournament info
        select m.status
        into old_status
        from matchup m
        where m.matchup_id = p_matchup_id;

        -- check if new status is passed in the param
        if old_status = p_new_status then
            select concat("Issue: Matchup id: ", p_matchup_id, " is already
in status '", p_new_status, "' ") as message;
```

```

else

    -- update the status
    update matchup
    set status = p_new_status
    where matchup_id = p_matchup_id;

    SELECT CONCAT('Matchup ', p_matchup_id, ' updated successfully!')
AS message;
    end if;
end if;

END //
DELIMITER ;

```

Command to execute procedure:

```
CALL update_matchup_status(49, 'Active');
```

```

MariaDB [fantasytourney]> select m.matchup_id, m.round_number, m.status,
-> t.title as tournament,
-> e1.name as entryA, e2.name as entryB
-> from matchup m
-> join tournament t on m.tournament_id = t.tournament_id
-> join entry e1 on m.entryA_id = e1.entry_id
-> join entry e2 on m.entryB_id = e2.entry_id
-> where m.matchup_id = 49;

```

matchup_id	round_number	status	tournament	entryA	entryB
49	3	Upcoming	Greatest Video Game	The Legend of Zelda: Ocarina of Time	Red Dead Redemption 2

```

1 row in set (0.000 sec)

```

```

MariaDB [fantasytourney]> select m.matchup_id, m.round_number, m.status,
-> t.title as tournament,
-> e1.name as entryA, e2.name as entryB
-> from matchup m
-> join tournament t on m.tournament_id = t.tournament_id
-> join entry e1 on m.entryA_id = e1.entry_id
-> join entry e2 on m.entryB_id = e2.entry_id
-> where m.matchup_id = 49;

```

matchup_id	round_number	status	tournament	entryA	entryB
49	3	Upcoming	Greatest Video Game	The Legend of Zelda: Ocarina of Time	Red Dead Redemption 2

```

1 row in set (0.000 sec)

MariaDB [fantasytourney]> CALL update_matchup_status(49, 'Active');

```

message
Matchup 49 updated successfully!

```

1 row in set (0.002 sec)

Query OK, 3 rows affected (0.003 sec)

MariaDB [fantasytourney]> select matchup_id, status, round_number
-> from matchup
-> where matchup_id = 49;

```

matchup_id	status	round_number
49	Active	3

```

1 row in set (0.000 sec)

```

4.3.4 *get_tournament_stats()* - procedure for statistical metrics

This procedure generates comprehensive statistical reports for a specific tournament, providing insights into participation rates, voting patterns, and entry popularity. This combines multiple complex queries into a single procedure call, making it easier to retrieve analytics on an entire tournament.

This procedure can serve multiple purposes; administrators can use it to monitor tournament engagement and identify popular entries, users can view tournament statistics on a result/summary page, and view their results or maybe compare their opinions/options with the overall outcome of how the tournament turned out.

```
DELIMITER //
```

```
CREATE PROCEDURE get_tournament_stats(  
    p_tournament_id int  
)  
BEGIN  
    DECLARE tournament_exists INT DEFAULT 0;  
  
    -- Check if tournament exists  
    select count(*) into tournament_exists  
    from tournament  
    where tournament_id = p_tournament_id;  
  
    if tournament_exists = 0 then  
        select 'Error: Tournament not found' as message;  
  
    else  
        -- Tournament Overview  
        select  
            t.tournament_id,  
            t.title,  
            t.description,  
            t.tourneystatus,  
            t.start_at,  
            t.end_at,  
            DATEDIFF(COALESCE(t.end_at, now()), t.start_at) as duration_days,  
            u.displayname as created_by_admin  
        from tournament t  
        join users u ON t.user_id = u.user_id  
        where t.tournament_id = p_tournament_id;  
  
        -- Participation Statistics  
        select  
            count(DISTINCT e.entry_id) as total_entries,  
            count(DISTINCT m.matchup_id) as total_matchups,  
            count(DISTINCT CASE WHEN m.status = 'Completed' THEN m.matchup_id  
END) as completed_matchups,  
            count(DISTINCT CASE WHEN m.status = 'Active' THEN m.matchup_id  
END) as active_matchups,  
            count(DISTINCT CASE WHEN m.status = 'Upcoming' THEN m.matchup_id  
END) as upcoming_matchups,  
            count(DISTINCT v.user_id) as unique_voters,
```

```

        count(v.user_id) as total_votes,
        round(count(v.user_id) / nullif(count(DISTINCT m.matchup_id), 0),
2) as avg_votes_per_matchup,
        max(m.round_number) as max_round_reached
    from tournament t
    left join entry e on t.tournament_id = e.tournament_id
    left join matchup m on t.tournament_id = m.tournament_id
    left join vote v on m.matchup_id = v.matchup_id
    where t.tournament_id = p_tournament_id;

-- Top 5 Most Popular Entries
select
    e.entry_id,
    e.name AS entry_name,
    e.seed,
    count(v.user_id) AS total_votes_received,
    count(DISTINCT m.matchup_id) AS matchups_participated
from entry e
left join vote v on e.entry_id = v.entry_id
left join matchup m on (e.entry_id = m.entryA_id OR e.entry_id =
m.entryB_id)
where e.tournament_id = p_tournament_id
group by e.entry_id, e.name, e.seed
order by total_votes_received desc
limit 5;

-- Vote Distribution by Round
select
    m.round_number,
    count(DISTINCT m.matchup_id) as matchups_in_round,
    count(v.user_id) as votes_in_round,
    round(count(v.user_id) / nullif(count(DISTINCT m.matchup_id), 0),
2) as avg_votes_per_matchup
from matchup m
left join vote v on m.matchup_id = v.matchup_id
where m.tournament_id = p_tournament_id
group by m.round_number
order by m.round_number;
end if;

END //

DELIMITER ;

```

Command to execute procedure:

```
CALL get_tournament_stats(2);
```

```

MariaDB [fantasytourney]> CALL get_tournament_stats(2);
+-----+-----+-----+-----+-----+-----+-----+-----+
| tournament_id | title           | description           | tourneystatus | start_at | end_at | duration_days | created_by_admin |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2             | Ultimate Marvel Hero | Which Marvel superhero reigns supreme? | Completed     | 2024-03-01 | 2024-04-01 | 31           | TourneyMaster    |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)

+-----+-----+-----+-----+-----+-----+-----+-----+
| total_entries | total_matchups | completed_matchups | active_matchups | upcoming_matchups | unique_voters | total_votes | avg_votes_per_matchup | max_round_reached |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 8             | 7             | 7                 | 0              | 0              | 16           | 288        | 41.14                | 3                 |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.002 sec)

+-----+-----+-----+-----+-----+
| entry_id | entry_name | seed | total_votes_received | matchups_participated |
+-----+-----+-----+-----+-----+
| 9        | Spider-Man | 1    | 39                   | 3                     |
| 11       | Captain America | 4    | 27                   | 3                     |
| 15       | Doctor Strange | 2    | 8                    | 2                     |
| 14       | Hulk        | 6    | 8                    | 2                     |
| 10       | Iron Man    | 8    | 2                    | 1                     |
+-----+-----+-----+-----+-----+
5 rows in set (0.005 sec)

+-----+-----+-----+-----+
| round_number | matchups_in_round | votes_in_round | avg_votes_per_matchup |
+-----+-----+-----+-----+
| 1            | 4                 | 19             | 4.75                  |
| 2            | 2                 | 9              | 4.50                  |
| 3            | 1                 | 8              | 8.00                  |
+-----+-----+-----+-----+
3 rows in set (0.005 sec)

Query OK, 1 row affected (0.005 sec)

```

4.4 - Triggers

In this section, we reviewed all foreign keys used in our schema and ensured that each constraint included appropriate ON UPDATE and ON DELETE behaviors. These rules help maintain referential integrity and simplify cascading deletions throughout the application.

4.4.1 - Foreign Key Constraints

Foreign key constraints enforce the relationships between tables in the Fantasy Tournament database and ensure that the system maintains referential integrity throughout all tournament-related operations. These constraints prevent invalid references, enforce parent/child rules between tables, and maintain consistency when data changes.

As part of Phase 4, we reviewed all existing CREATE TABLE statements from Phase 3 and confirmed that all foreign key relationships have already been implemented correctly. These constraints ensure predictable behavior when tournaments, entries, matchups, and votes are inserted, updated, or deleted.

Below are the foreign key constraints currently enforced in our schema:

1. Users → Tournament (RESTRICT)

Each tournament must belong to a valid user. Because this FK uses the default ON DELETE/ ON UPDATE RESTRICT, a user cannot be deleted while they still have tournaments associated with them. This prevents “ownerless” tournaments and reflects realistic behavior.

SQL Used:

foreign key (user_id) references users(user_id)

2. Tournament → Entry (RESTRICT)

Every entry must belong to an existing tournament. The RESTRICT behavior prevents deleting a tournament while its entries still exist. This protects the integrity of tournament content and prevents accidental loss of entire sets of entries.

SQL Used:

```
foreign key (tournament_id) references tournament(tournament_id)
```

3. Tournament/Entry → Matchup (RESTRICT)

Matchups depend on both a tournament and two valid entries. The default RESTRICT prevents deleting a tournament if matchups exist and deleting an entry if it is used in any matchup. This prevents matchups from becoming invalid.

SQL Used:

```
foreign key (tournament_id) references tournament(tournament_id),  
foreign key(entryA_id) references entry(entry_id),  
foreign key(entryB_id) references entry(entry_id)
```

4. Users/Matchup/Entry → Vote (RESTRICT)

Votes depend on a valid user, a valid matchup, and a valid entry. All foreign keys use RESTRICT, meaning a matchup cannot be deleted, an entry cannot be deleted if someone voted for it, and a user cannot be deleted if they have cast votes. This preserves the complete voting history. Deleting user accounts will need to be revised and adjusted in Phase 5 to allow the deleted account's data to persist after the account has been deleted.

SQL Used:

```
foreign key (user_id) references users(user_id),  
foreign key (matchup_id) references matchup(matchup_id),  
foreign key (entry_id) references entry(entry_id)
```

4.4.2 - Explicit Triggers

4.4.2.1 trg_entry_same_ins & trg_entry_same_upd

These two triggers work together to strictly enforce unique entry names within a single tournament. trg_entry_same_ins (INSERT) ensures that when a new entry is added, its name does not conflict with an existing entry in the same tournament. trg_entry_same_upd

(UPDATE) ensures that when an existing entry is renamed, it is not changed to a name that is already in use by another competitor in that tournament. Together, these triggers protect the integrity of the tournament bracket. If duplicate names were allowed, the GUI visualization would become confusing (e.g., users wouldn't know which "Team A" they were voting for), potentially corrupting the voting results.

```
DELIMITER //
```

```
DROP TRIGGER IF EXISTS trg_entry_same_ins //
```

```
CREATE TRIGGER trg_entry_same_ins
```

```
BEFORE INSERT ON entry
```

```
FOR EACH ROW
```

```
BEGIN
```

```
  IF EXISTS (
```

```
    SELECT 1
```

```
    FROM entry e
```

```
    WHERE e.tournament_id = NEW.tournament_id
```

```
          AND e.name = NEW.name
```

```
  ) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Duplicate entry name in this tournament.';
```

```
  END IF;
```

```
END //
```

```
DROP TRIGGER IF EXISTS trg_entry_same_upd //
```

```
CREATE TRIGGER trg_entry_same_upd
```

```
BEFORE UPDATE ON entry
```

```
FOR EACH ROW
```

```
BEGIN
```

```
  IF EXISTS (
```

```
    SELECT 1
```

```
    FROM entry e
```

```
    WHERE e.tournament_id = NEW.tournament_id
```

```
          AND e.name = NEW.name
```

```
          AND e.entry_id <> NEW.entry_id
```

```
  ) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Duplicate entry name in this tournament.';
```

```
  END IF;
```

```
END //
```

trg_entry_same_ins:

```
MariaDB [fantasytourney]> insert into entry (name, description, image_url, seed, tournament_id)
-> VALUES('Spider-Man', 'dupe', NULL, 99, 2);
ERROR 1644 (45000): Duplicate entry name in this tournament.
MariaDB [fantasytourney]> select entry_id,name, seed, tournament_id
-> from entry
-> where tournament_id = 2
-> order by entry_id;
```

entry_id	name	seed	tournament_id
9	Spider-Man	1	2
10	Iron Man	8	2
11	Captain America	4	2
12	Thor	5	2
13	Black Widow	3	2
14	Hulk	6	2
15	Doctor Strange	2	2
16	Black Panther	7	2

8 rows in set (0.001 sec)

trg_entry_same_upd:

```
MariaDB [fantasytourney]> update entry
-> set name = 'Spider-Man'
-> where entry_id = 10
-> ;
ERROR 1644 (45000): Duplicate entry name in this tournament.
MariaDB [fantasytourney]> select entry_id, name, tournament_id
-> from entry
-> where tournament_id = 2
-> and entry_id in (9,10);
```

entry_id	name	tournament_id
9	Spider-Man	2
10	Iron Man	2

2 rows in set (0.000 sec)

4.4.2.2 trg_matchup_min_votes_upd

This trigger implements a "Quality Control" rule for our tournaments. Its purpose is to ensure that a matchup cannot be finalized unless it has received at least a minimal level of activity from users (set here to 1 vote). If an administrator or the system attempts to mark a matchup as 'Completed' while the vote count is zero, the trigger raises an exception. This is intended to prevent tournaments from advancing automatically with "empty" results, encouraging administrators to wait for user activity before closing a round. (Tournament no longer needs a minimum number of votes as the lowest seed always advances)

```
DELIMITER //
```

```
DROP TRIGGER IF EXISTS trg_matchup_min_votes_upd //
```

```

CREATE TRIGGER trg_matchup_min_votes_upd
BEFORE UPDATE ON matchup
FOR EACH ROW
BEGIN

IF NEW.status = 'Completed' AND OLD.status <> 'Completed' THEN
  IF (SELECT COUNT(*) FROM vote WHERE matchup_id = NEW.matchup_id) < 1 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot complete matchup: minimum vote threshold not met.';
  END IF;
END IF;
END //

DELIMITER ;

```

```

MariaDB [fantasytourney]> SELECT COUNT(*) AS votes_for_1783
-> FROM vote
-> WHERE matchup_id = 1783;

```

```

+-----+
| votes_for_1783 |
+-----+
|                0 |
+-----+

```

```

1 row in set (0.000 sec)

```

```

MariaDB [fantasytourney]>
MariaDB [fantasytourney]> UPDATE matchup
-> SET status = 'Active',
-> winner_entry_id = NULL
-> WHERE matchup_id = 1783;

```

```

Query OK, 0 rows affected (0.000 sec)
Rows matched: 1 Changed: 0 Warnings: 0

```

```

MariaDB [fantasytourney]>
MariaDB [fantasytourney]> UPDATE matchup
-> SET status = 'Completed'
-> WHERE matchup_id = 1783;
ERROR 1644 (45000): Cannot complete matchup: minimum vote threshold not met.

```

4.4.2.3 trg_matchup_select_winner

This trigger is designed to automate the business logic of determining a match winner. Instead of writing complex application code to query vote counts, compare them, handle tie-breakers, and then issue a separate update, this trigger handles it all atomically within the database. We intend to use this in the Phase 5 "Round Advancement" logic. When a matchup's time limit expires, the application will simply update the status to 'Completed', and this trigger will instantly calculate the winner based on votes (or seed tie-breakers) and update the winner_entry_id. This guarantees that a completed match never exists without a declared result.

```

DELIMITER //

DROP TRIGGER IF EXISTS trg_matchup_select_winner //
CREATE TRIGGER trg_matchup_select_winner
BEFORE UPDATE ON matchup
FOR EACH ROW
BEGIN
    DECLARE votesA INT DEFAULT 0;
    DECLARE votesB INT DEFAULT 0;
    DECLARE seedA INT DEFAULT NULL;
    DECLARE seedB INT DEFAULT NULL;

    IF NEW.status = 'Completed' AND OLD.status <> 'Completed' THEN

        SELECT COUNT(*)
        INTO votesA
        FROM vote
        WHERE matchup_id = NEW.matchup_id
        AND entry_id = NEW.entryA_id;

        SELECT COUNT(*)
        INTO votesB
        FROM vote
        WHERE matchup_id = NEW.matchup_id
        AND entry_id = NEW.entryB_id;

        IF votesA > votesB THEN

            SET NEW.winner_entry_id = NEW.entryA_id;

        ELSEIF votesB > votesA THEN

            SET NEW.winner_entry_id = NEW.entryB_id;

        ELSE
            SELECT seed
            INTO seedA
            FROM entry
            WHERE entry_id = NEW.entryA_id;

            SELECT seed
            INTO seedB
            FROM entry
            WHERE entry_id = NEW.entryB_id;

            SET seedA = COALESCE(seedA, 9999);
            SET seedB = COALESCE(seedB, 9999);

            IF seedA < seedB THEN

```

```

        SET NEW.winner_entry_id = NEW.entryA_id;
    ELSEIF seedB < seedA THEN
        SET NEW.winner_entry_id = NEW.entryB_id;
    ELSE
        SET NEW.winner_entry_id = NEW.entryA_id;
    END IF;

END IF;

END IF;
END //

DELIMITER ;

```

```

MariaDB [fantasytourney]> SELECT matchup_id, status, entryA_id, entryB_id, winner_entry_id
-> FROM matchup
-> WHERE matchup_id = 1783;
+-----+-----+-----+-----+-----+
| matchup_id | status | entryA_id | entryB_id | winner_entry_id |
+-----+-----+-----+-----+-----+
|      1783 | Active |      1885 |      1886 |             NULL |
+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [fantasytourney]> UPDATE matchup
-> SET status = 'Completed'
-> WHERE matchup_id = 1783;
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [fantasytourney]> SELECT matchup_id, status, entryA_id, entryB_id, winner_entry_id
-> FROM matchup
-> WHERE matchup_id = 1783;
+-----+-----+-----+-----+-----+
| matchup_id | status   | entryA_id | entryB_id | winner_entry_id |
+-----+-----+-----+-----+-----+
|      1783 | Completed |      1885 |      1886 |             1885 |
+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

```

4.4.2.3 Trg_entry_prevent_delete_active

This trigger acts as a critical safety lock for our data integrity. Once a tournament is marked as 'Active', the bracket structure relies on specific entries being present to link matchups together. Deleting an entry during a live tournament would corrupt the data and likely break the visualization in our future GUI. This trigger is intended to prevent administrators or accidental code execution from removing a competitor while the tournament is ongoing. It forces the tournament to be set to 'Completed' or 'Upcoming' before any structural changes can be made.

```

DELIMITER //

CREATE TRIGGER trg_entry_prevent_delete_active
BEFORE DELETE ON entry

```

```

FOR EACH ROW
BEGIN
    DECLARE tourney_status VARCHAR(20);

    -- Check the status of the tournament this entry belongs to
    SELECT tourneystatus INTO tourney_status
    FROM tournament
    WHERE tournament_id = OLD.tournament_id;

    -- If tournament is active, block the delete
    IF tourney_status = 'Active' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Safety Lock: Cannot delete an entry while the
tournament is Active.';
    END IF;
END //

DELIMITER ;

```

```

MariaDB [fantasytourney]> delete from entry
-> where entry_id = 25;
ERROR 1644 (45000): Safety Lock: Cannot delete an entry while the tournament is Active.
MariaDB [fantasytourney]> SELECT
->     e.entry_id,
->     e.name,
->     e.tournament_id,
->     t.tourneystatus
-> FROM entry e
-> JOIN tournament t
->   ON e.tournament_id = t.tournament_id
-> WHERE t.tourneystatus = 'Active'
-> LIMIT 5;

```

entry_id	name	tournament_id	tourneystatus
25	The Matrix	4	Active
26	Pulp Fiction	4	Active
27	Jurassic Park	4	Active
28	The Shawshank Redemption	4	Active
29	Forrest Gump	4	Active

```

5 rows in set (0.000 sec)

```

Phase 5 – Database Programming and Graphical User Interface (GUI) Application Development

5.0 - GUI Development

The Fantasy Tourney application supports three user groups: **Guests**, **Players**, and **Admins**.

- **Guests** have read-only access to browse active/upcoming tournaments and view tournament brackets. They cannot vote or create tournaments.
- **Players** are the main users who can create tournaments, vote in active matchups, and manage their personal profiles.
- **Admins** possess overall supervision, can moderate tournaments, delete tournaments/entries, and delete users.

5.0.1 Examples of CRUD Operations Supported in the GUI

Each user group performs different create, read, update, and delete operations:

Create:

Registering a new account to become a member (guest), and then he becomes a registered player who can create tournaments with 16 entries (player, admin), and can cast votes on matchups when tournaments are active (player, admin).

Read:

Browsing tournaments and their respective brackets (Guest, Player, Admin). Viewing the full tournament statistics page (Player, Admin). Tournaments created, entries voted on, and participation can be viewed on the profile page (Player). All players and tournaments on the site can be viewed on the Admin panel (Admin).

Update:

Update profiles, inclusive of unique username, bio, profile picture, and password under the profile page (Player). Update tournament settings like start & end datetimes (Player). Update Tournament statuses, making tournaments end early (Admin).

Delete:

Delete the tournaments they created (Player). Delete any tournament or user (Admin).

Report Generation

The application generates a detailed statistical report for any tournament, including tournament details, vote distribution by each round in tournaments, participation metrics, and "Top 5 Entries". This report is generated using aggregate functions via the `get_tournament_stats` stored procedure and can be saved as a PDF using the button provided in the tournament stats page, where we use the browser's native print to PDF option to locally save the tournament statistics as it is on the webpage.

5.0.2 Use of Database Programming in the GUI

The GUI extensively utilizes these forms of database logic handling validated, consistent data and prevents unintended or invalid operations.

1. **Views** are implemented to simplify complex read operations; for instance, the *view_active_tournaments* and *view_tournament_summary* views allow the browsing and admin pages to load tournament lists without executing repetitive, complex JOIN queries in the application layer.
2. **Stored Procedures** manage all atomic transactions: user registration is handled by *register_user*, tournament creation by *create_tournament*, and safe deletion by *delete_tournament*. Finally, the detailed statistical reports on the "Statistics" page are generated entirely by the *get_tournament_stats* procedure, which aggregates vote and matchup data on the server side.
3. **Triggers** help maintain data integrity, on which the GUI indirectly relies. Preventing the insertion of duplicate entry names within the same tournament is done with *Trg_entry_same_ins* and *Trg_entry_same_upd*, ensuring tournament creators cannot create two entries with the same name, preventing bracket inconsistencies.

5.0.3 Database Tables Used

Directly Used:

The GUI directly uses **users** table for authentication and profile management. The **tournament** and **entry** tables are accessed directly to render the bracket visualization in the player interface and to list out all the tournaments with respect to their statuses. The **matchup** and **vote** tables are heavily utilized during the voting process to record user choices, to progress through each round, show individual player votes, and to display real-time results.

Indirectly Used:

Through views and stored procedures that aggregate or filter tournament information, we do complex aggregations of the **vote** and **matchup** tables are abstracted via views such as *active_matchups_with_votes*, *view_tournament_entries*, and *view_tournament_summary*. These views allow the Admin Panel and Tournament List to load summaries and status information without executing complex JOINS in the application code. Similarly, the *get_tournament_stats* stored procedure encapsulates heavy aggregation logic for the Statistics page, which allows the application layer to retrieve a full statistical report using a single, simple stored procedure call rather than making multiple queries exposing raw table logic to the front end.

5.1 - GUI Functionalities and User Groups

The Fantasy Tournament GUI supports three user groups: **Guests**, **Players**, and **Admins**. Each user group interacts with the application differently based on the permissions defined in the database and enforced through the GUI.

Guests have read-only access to public tournament information. They can browse active and upcoming tournaments and view their brackets, but cannot vote or create tournaments.

Players have full participation access. They can create tournaments, vote in matchups, customize their profiles, and view tournament statistics. Players can also track tournaments they created or voted in.

Admins inherit all Player capabilities but also have access to moderation tools. Admins can delete tournaments or users, end tournaments early, and view administrative data such as all tournaments and all users.

The Fantasy Tournament GUI adapts dynamically based on the logged-in user's role, ensuring that each group sees only the controls appropriate for their access level.

5.1.1 - User Capabilities

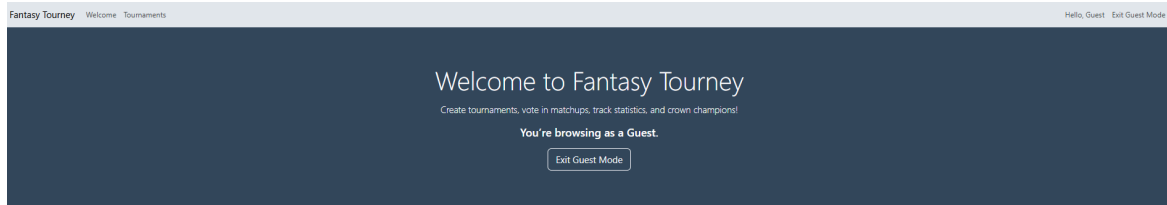
Guest Capabilities- Unregistered visitors are restricted to a "Read-Only" mode. They can browse the tournament feed to view lists of active, upcoming, and completed tournaments and inspect the brackets of a specific tournament to see live results. However, functionality that requires writing to the database—such as voting or creating tournaments—is disabled, and attempting to access these features redirects them to a "Access Denied" page prompting registration.

Player Capabilities- This is the primary user group and has full participation access. Upon logging in, players gain the ability to interact with the database by voting in active matchups during active rounds. They can access a private Profile Dashboard where they can view their voting history, edit their account details (Display Name, Bio, profile picture), and manage tournaments they have created. The "Create Tournament" interface is exclusive to logged-in users, allowing them to upload images, set start and end dates, and define entries for a new bracket.

Admin Capabilities- Admins inherit all Player capabilities but have exclusive access to the Admin Panel. This interface allows them to search for any user or tournament. Admins serve a moderation role; they can permanently delete any user or tournament that violates platform rules. These deletion actions cascade through the database to maintain referential integrity.

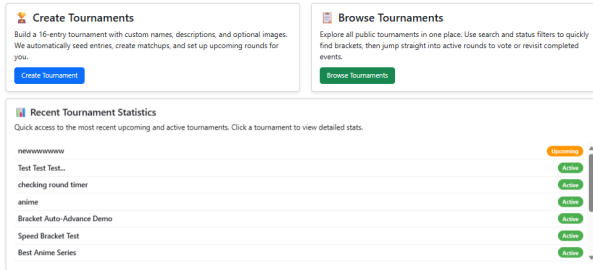
5.1.2 - Screenshots and Walkthrough

Guest Experience-

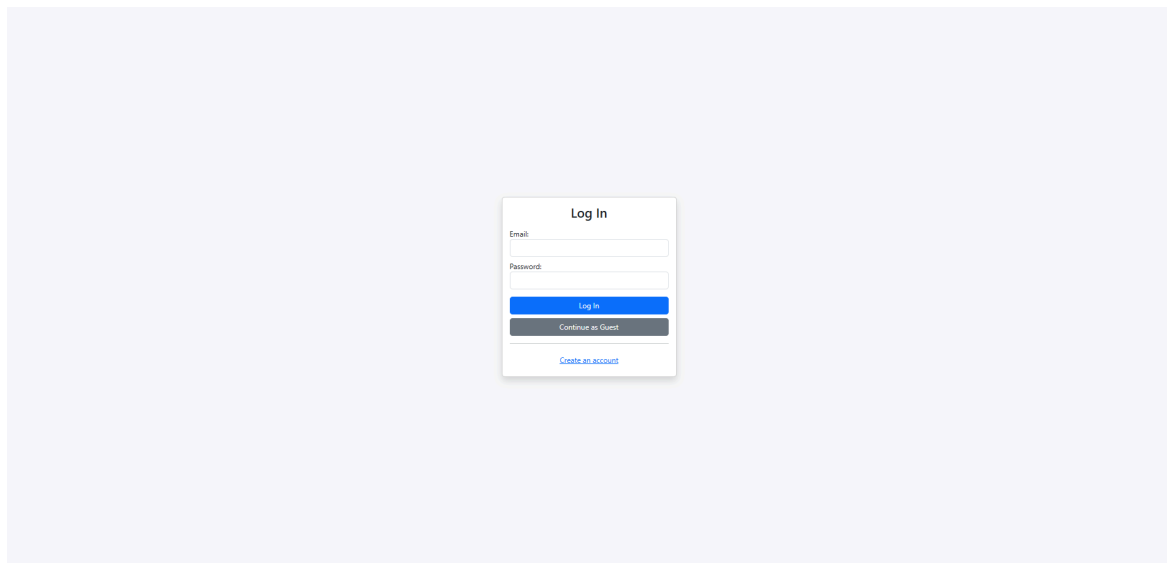


About the Platform

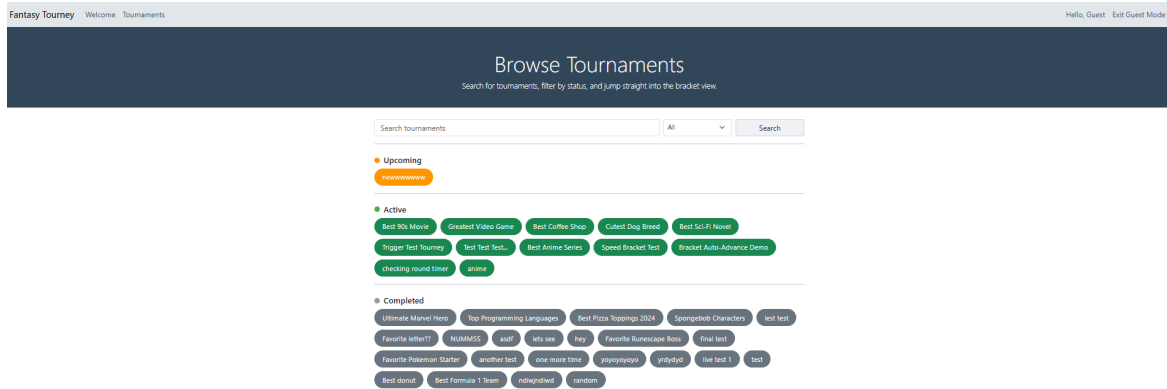
Fantasy Tourney lets you build single-elimination brackets, upload entry images, run timed matchups, vote live, and explore tournament statistics.



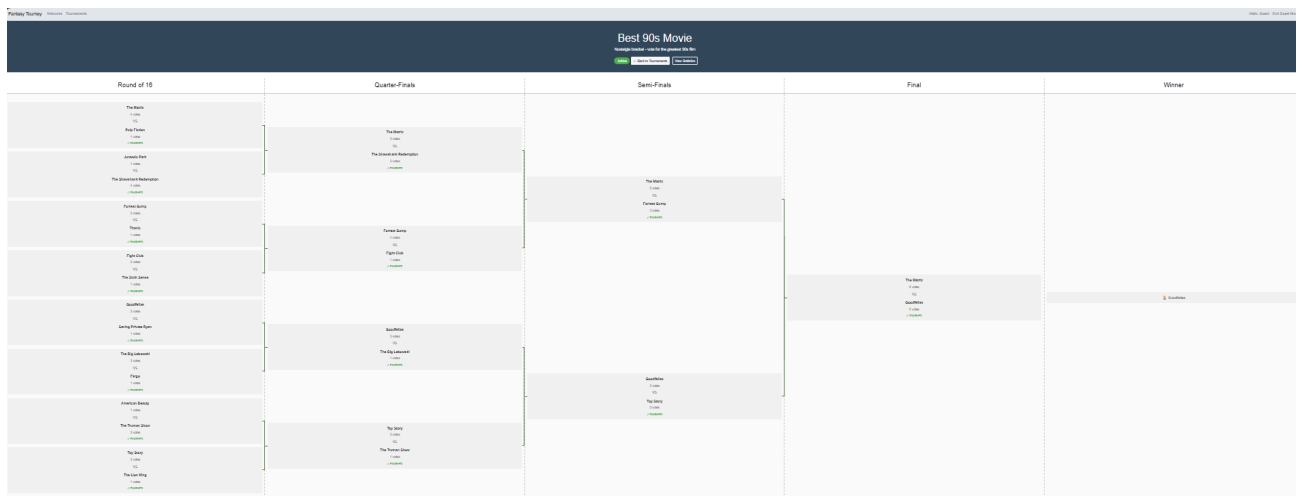
When a user first visits Fantasy Tourney without logging in, they are treated as a Guest. Our navigation bar at the top recognizes this, as does the welcome header, greeting them appropriately. The user is prompted with the option to “Exit Guest Mode.” If they click on that, they are taken to the log in page where they can either log in, continue as a guest, or create an account.



Within the welcome page, guest users are able to view a quick description of Fantasy Tourney: “Create tournaments, vote in matchups, track statistics, and crown champions!” and another small section about the platform: “Fantasy Tourney lets you build single-elimination brackets, upload entry images, run timed matchups, vote live, and explore tournament statistics.” From the three options available on the Welcome page, guest users can access only two, “Browse Tournaments” and “Recent Tournament Statistics,” as attempting to select the “Create Tournament” option will take you to the login page.



From the browse tournaments page, users are able to view the names of tournaments and their status: Active, Upcoming, or Completed. The tournament names are also color-coded to further make their status clear—Upcoming is orange, Active is green, and Completed is grey. From this page, users can also search for specific tournaments and filter tournaments by their status.



Users are also able to click on a tournament and view the brackets. They can see voting as it is happening, but they are not able to participate. Within this page, they can also access the statistics page, which we will mention momentarily. Clicking on “Fantasy Tourney” in the top-left corner takes users back to the welcome page. We will now explore the other option: guest users also have access to “Recent Tournament Statistics.”

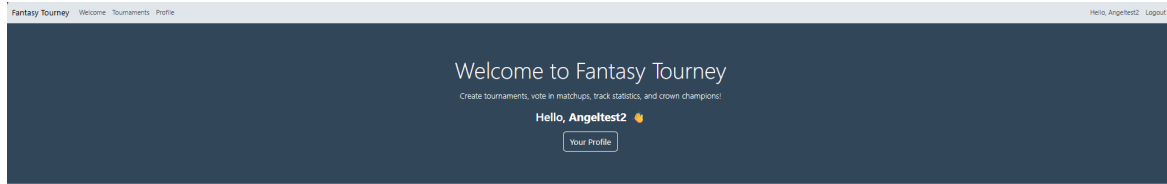
Quick

ne

at

Early Tourist

1a.



About the Platform

Fantasy Tourney lets you build single-elimination brackets, upload entry images, run timed matchups, vote live, and explore tournament statistics.

Create Tournaments

Build a 16-entry tournament with custom names, descriptions, and optional images. We automatically seed entries, create matchups, and set up upcoming rounds for you.

Create Tournament

Browse Tournaments

Explore all public tournaments in one place. Use search and status filters to quickly find brackets, then jump straight into active rounds to vote or revisit completed events.

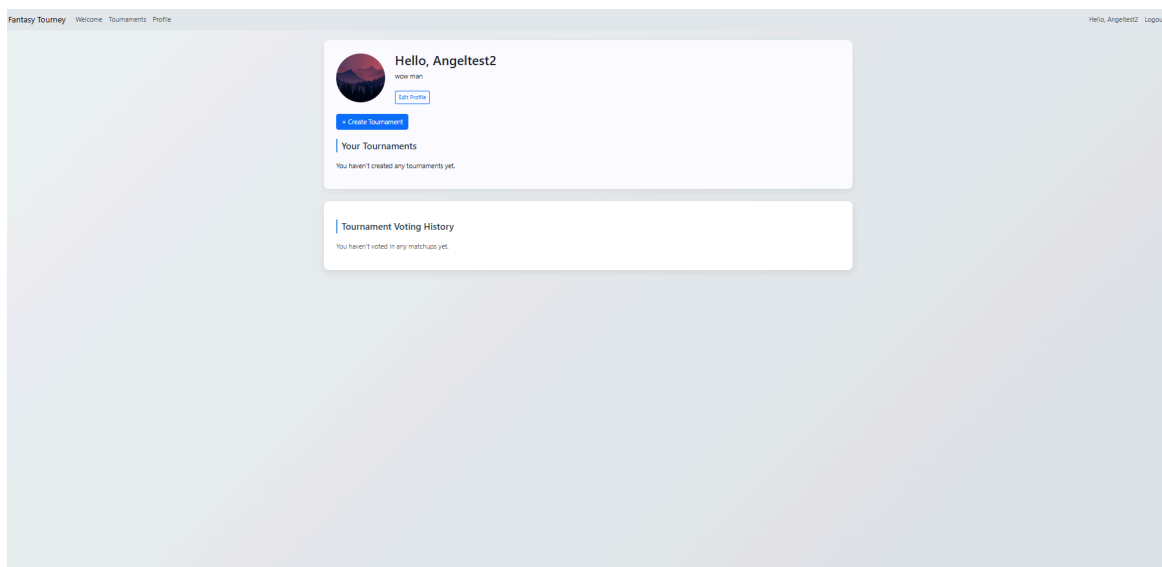
Browse Tournaments

Recent Tournament Statistics

Quick access to the most recent upcoming and active tournaments. Click a tournament to view detailed stats.

newwwwwww	Upcoming
Test Test Test...	Active
checking round timer	Active
anime	Active
Bracket Auto-Advance Demo	Active
Speed Bracket Test	Active
Best Anime Series	Active

This is the view when a user is logged in as a Player. It is, for the most part, the same as the guest view; however, as a Player, the Create Tournaments page is now accessible, and users are able to access their profile, which we will look at first.



From this page, a user is able to view their participation in tournaments via their voting history and their own created tournaments, as well as access the edit profile page, where they can do things like edit their profile picture, bio, username, and password.

Edit Profile

Username



Bio

Change Profile Picture (optional)

Choose File | No file chosen

Change Password

Current Password

Enter current password to change

New Password

Enter new password

Confirm New Password

Confirm new password

[Save Changes](#)

[Cancel](#)

Now we will go ahead and go to the Create Tournaments page by clicking on “Fantasy Tourney” in the top left corner so we can access it via the Welcome page, although you are also able to access it from the profile page.

Create Tournament

Title:

Description:

Start Date/Time:

mm/dd/yyyy -- --

End Date/Time:

mm/dd/yyyy -- --

Entries (16 required)

Entry 1

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 2

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 3

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 4

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 5

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 6

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 7

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 8

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 9

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 10

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 11

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 12

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 13

Name:

Description (optional):

Image (optional):

Choose File No file chosen

Entry 14

Name:

Description (optional):

Image (optional):

Choose File No file chosen

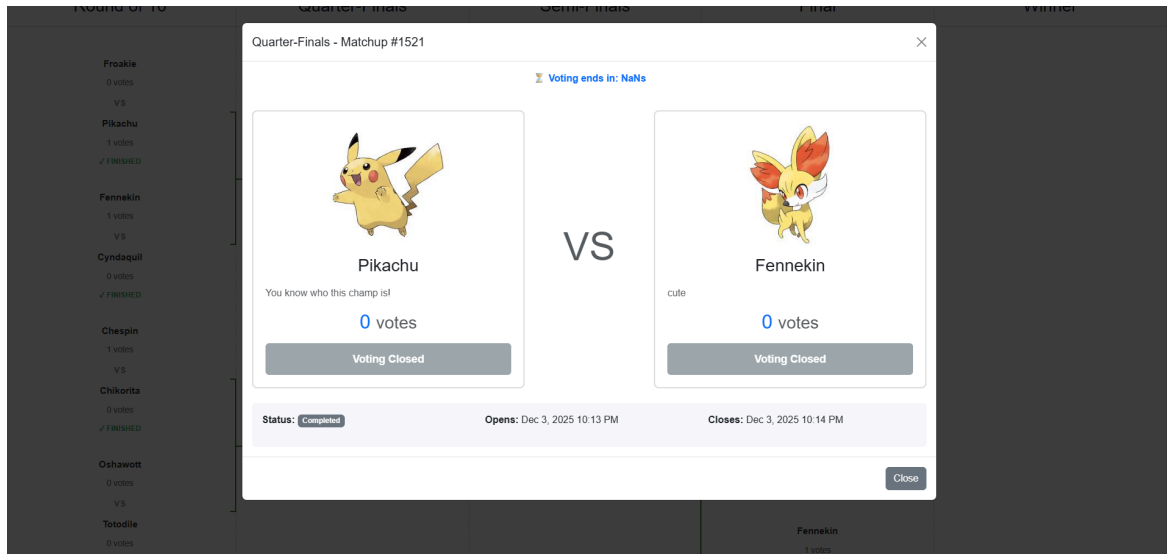
Entry 15

Name:

Entry 16

Name:

From this page, you can go ahead and create a tournament. You will need to insert 16 entries, with the description and image being optional in the creation of the tournament



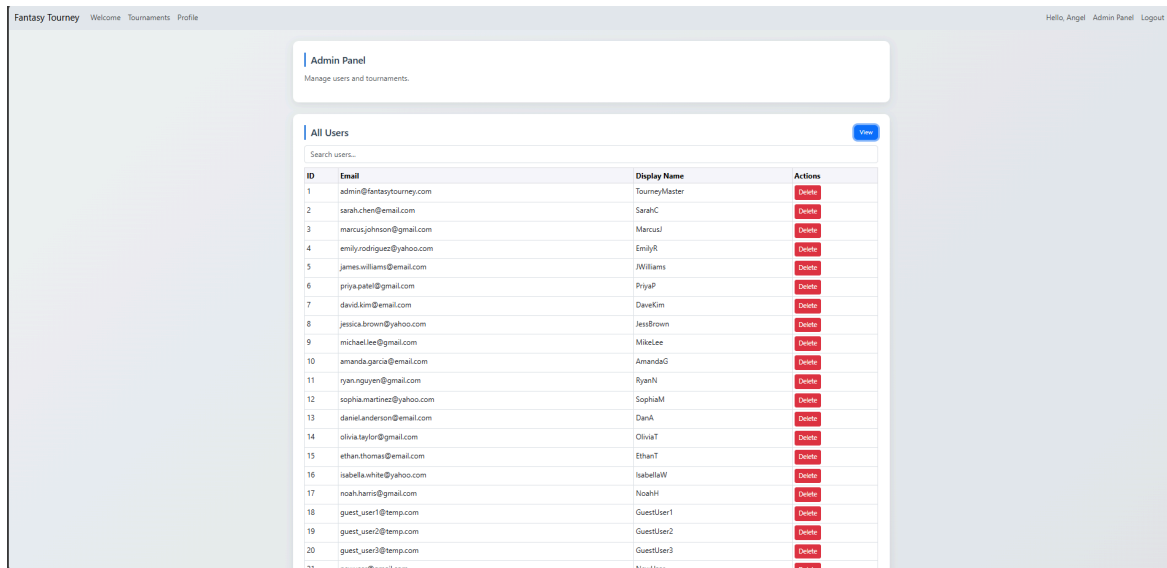
The most critical interaction for a Player is the ability to vote. When viewing an "Active" tournament bracket, Players are presented with interactive "Vote" buttons on current matchups. Clicking a button casts their vote for that specific entry. The interface provides visual feedback to confirm the vote has been recorded and updates the local view. The bracket also handles the round timer; once the countdown reaches zero, the page auto-refreshes to display the calculated winners, allowing the Player to see the results of their participation immediately.

Admin Experience-

The Admin role inherits all Player capabilities but gains access to the restricted Admin Panel via the navigation bar.

This dashboard provides a high-level overview of the platform's data. Administrators possess elevated privileges to Delete any user or tournament to enforce community standards (e.g., removing inappropriate content). Note that Admin privileges are currently assigned directly via the database backend (MariaDB) rather than through the GUI, ensuring strict access control.

Hello, Angel Admin Panel Logout



From the admin panel, admins have the ability to delete users and tournaments for whatever reason that may be, including but not limited to inappropriate behavior or harassment. Admins are assigned the role via MariaDB, and currently, there is no way to assign the role through the GUI.

5.2 - GUI Programming

5.2.1 - Database Layer Programming

The database layer consists of all server-side logic executed directly in MariaDB. This layer includes stored procedures, views, and triggers that support our GUI by enforcing business rules, simplifying queries, and maintaining data integrity. Each component below describes its purpose, where it is used in the application, and which user groups it serves.

5.2.1.1 Stored Procedures

1. register_user

This procedure validates and inserts new users into the 'users' table. It checks for missing fields, enforces valid roles (admin, player, guest), and prevents duplicate emails before creating the account. If validation passes, the user is inserted, and a confirmation message is returned.

This procedure is invoked by register.php. When a guest submits the registration form, the PHP script executes the SQL command CALL register_user(...). The procedure returns a success or error message (e.g., "Email already exists"), which the GUI displays to the user. This serves Guests and Players (registering an account).

2. delete_tournament

Safely deletes a tournament only if it is not active and contains no active matchups. The procedure checks tournament existence, ensures it is not currently running, removes all related votes, matchups, and entries, and finally deletes the tournament record.

This is invoked by `delete_tournament.php`, which is triggered when a user clicks the "Delete" button on the Profile Dashboard (for tournament owners) or the Admin Panel (for Admins). The PHP script passes the Tournament ID and the User's Role to the procedure to enforce permission checks. The user groups served with this procedure would be Admins (full deletion rights) and Players (limited use — can only delete their own tournaments before they start).

3. `get_tournament_stats`

Generates a complete statistical report for a tournament. This includes tournament overview, total entries, matchups, votes, and a unique voter count. It also produces the average votes per matchup, the top 5 most popular entries, and the vote distribution by round into a single procedure call, avoiding the need for multiple expensive queries from the client. This data powers the GUI's tournament statistics page.

This is used in `statistics.php`, the admin dashboard, and player-created tournament insights. The user groups served would be Players (viewing stats of tournaments they created or participated in) and Admins.

5.2.1.2 Views

1. `active_matchups_with_votes`

Provides a consolidated view of all active matchups, including entry names and images, vote counts for each entry, total votes, and round number and closing times.

This is used in the voting and results page. The user groups served Players (viewing active matchups).

2. `view_tournament_entries`

Simplifies retrieving all entries for a tournament with their names and images. Used to display tournament bracket contents, entry details, and statistics pages.

This is used in bracket rendering and the statistics display in `statistics.php`, displaying the full set of entries (with images) for a specific tournament. The user groups served with this would be Guests, Players, and Admins.

3. `view_active_tournaments`

Returns all tournaments with status 'Active'. This simplifies the main browsing query by pre-filtering the tournament status, making it easier for the GUI to show what users can vote in.

Utilized by tournaments.php to render the "Active" and "Upcoming" tournament lists without requiring the frontend code to understand the underlying status logic.. This serves all user groups.

5.2.1.3 Triggers

1. trg_entry_same_ins and trg_entry_same_upd

Prevents duplicate entry names within the same tournament. If a user attempts to insert or rename an entry to a name that already exists in that tournament, the trigger blocks the operation and returns an error. This ensures bracket integrity and avoids confusing matchups.

These triggers fire automatically during the execution of create_tournament.php. If a user attempts to upload duplicate entries, the database raises an error, which the GUI catches and reports to the user. The user groups served would be Players (prevents accidental duplicate entries).

2. trg_matchup_select_winner

Automatically calculates the winning entry whenever a matchup status changes to "Completed." It counts votes and applies a seed-based tie-breaker if votes are tied or zero, ensuring every completed matchup has a valid winner.

This trigger fires immediately when config.php executes the SQL command UPDATE matchup SET status='Completed'. By moving this logic to a trigger, we eliminated redundant PHP calculation loops. The user groups served would be all users/visitors(Guest, Player, Admin) who see the tournaments.

5.2.1. 4 New Programming Logic for Phase 5:

Stored Procedures

1. create_tournament

To support the GUI's 'Create Tournament' function, we created a new procedure that handles the initialization of a tournament record in a single step. It accepts the tournament entry data (title, description, start-time, end-time, user_id) and returns the newly created tournament_id.

Called by create_tournament.php. The returned ID is immediately used by the PHP script to associate the 16 uploaded entry images with the correct tournament.

Code used to create:

```

DROP PROCEDURE IF EXISTS create_tournament;

DELIMITER //

CREATE PROCEDURE create_tournament(
    IN p_title          VARCHAR(255),
    IN p_description    TEXT,
    IN p_start          DATE,
    IN p_end            DATE,
    IN p_user_id        INT
)
BEGIN
    DECLARE v_status VARCHAR(20);

    -- Decide tournament status based on dates
    IF p_start > CURDATE() THEN
        SET v_status = 'Upcoming';
    ELSEIF p_end IS NOT NULL AND p_end < CURDATE() THEN
        SET v_status = 'Completed';
    ELSE
        SET v_status = 'Active';
    END IF;

    -- Insert the new tournament
    INSERT INTO tournament (title, description, start_at, end_at,
    tourneystatus, user_id)
    VALUES (p_title, p_description, p_start, p_end, v_status, p_user_id);

    -- Return the new tournament_id as a result set
    SELECT LAST_INSERT_ID() AS new_tournament_id;
END //

DELIMITER ;

```

5.2.2 - Application Layer Programming

The application layer is implemented in PHP, running on Apache on the university's Artemis server. This layer serves as the bridge between the GUI (HTML/CSS/JavaScript) and the database layer (MariaDB). It manages establishing secure database connections, executing SQL queries, views, and stored procedures, and also handles authentication and session states. The application layer also enforces semantic constraints not guaranteed by the database, validates and sanitizes user inputs, and formats query results for display in the GUI.

Below are the key examples of PHP code that implement this functionality.

5.2.2.1 Database Connection (config.php)

All PHP files load a shared PDO connection through config.php where we centralize the PDO connection and instead of creating new connection in every single file we just make use of the static variable from the `get_pdo_connection()` function .

```

<?php
define("DB_HOST", "localhost");
define("DB_NAME", "fantasytourney");
define("DB_USER", "jsoto2");
define("DB_PASS", "*****");

function get_pdo_connection() {
    $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME;
    $pdo = new PDO($dsn, DB_USER, DB_PASS);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    return $pdo;
}
?>

```

This provides a secure, centralized database connection, enables prepared statements, and ensures consistent error handling across all PHP pages.

5.2.2.2 Executing a Stored Procedure (register_user)

Register.php calls the stored procedure using prepared statements:

```

$stmt = $db->prepare("CALL register_user(?, ?, ?, ?)");
$stmt->execute([$email, $password, $displayname, $role]);
$result = $stmt->fetchAll(PDO::FETCH_ASSOC);

```

This sends form data to MariaDB, receives DB-generated success/error messages, and ensures proper parameter binding.

5.2.2.3 Deleting a Tournament via Stored Procedure (delete_tournament.php)

Allows admins to delete tournaments while relying on the stored procedure to enforce rules (no deletion of active tournaments, safe cascading deletes).

```

$tournament_id = $_POST['tournament_id'];

$stmt = $db->prepare("CALL delete_tournament(?)");
$stmt->execute([$tournament_id]);

$message = $stmt->fetchColumn();
echo "<p>$message</p>";

```

5.2.2.4 Using a View to Fetch Active Matchups (admin.php)

This retrieves matchup + vote data from a multi-join view and simplifies PHP logic. It is used for moderation and live status display.

```

$query = $db->query("SELECT * FROM active_matchups_with_votes ORDER BY
closes_at ASC");

```

```
$activeMatchups = $query->fetchAll(PDO::FETCH_ASSOC);
```

Loads all live matchups using a database view that handles complex JOIN logic. Simplifies PHP and supports live moderation.

5.2.2.5 Executing Direct Queries (tournaments.php)

Retrieving all tournaments via prepared statements:

```
$stmt = $db->prepare("SELECT * FROM tournament ORDER BY start_at DESC");  
$stmt->execute();  
$tournaments = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Retrieves tournament lists for the main browsing page.

5.2.2.6 Calling get_tournament_stats (statistics.php)

And, an example of how we display the results we get from the get_tournament_stats procedure from the statistics.php page is, the procedure returns distinct result sets (like overview, participation, top-entries, and vote distribution) in a single call. The PHP application layer handles this using nextRowset(), where we iterate through the result data effectively and deliver it without needing 4 separate database queries

```
$query = $db->prepare("CALL get_tournament_stats(:tid)");  
$query->bindParam(":tid", $tournamentId, PDO::PARAM_INT);  
$query->execute();  
  
// RESULT SET: OVERVIEW  
$overview = $query->fetchAll(PDO::FETCH_ASSOC);  
  
// RESULT SET: PARTICIPATION  
$query->nextRowset();  
$participation = $query->fetch(PDO::FETCH_ASSOC) ?: [];  
  
// RESULT SET: VOTE DISTRIBUTION  
$query->nextRowset();  
$roundStats = $query->fetchAll(PDO::FETCH_ASSOC);
```

5.2.2.7 Semantic Constraint Enforcement in PHP (cast_vote.php)

Preventing double voting in the same matchup. Although the database has a Primary Key on (user_id, matchup_id) that effectively prevents double voting, relying solely on the database would result in a fatal SQL error. Instead, the application layer pre-checks if a vote exists. This allows us to provide a clean, user-friendly error message ("You have already voted") rather than crashing the script.

```
// Check if user already voted on this matchup  
$voteCheck = $db->prepare(" SELECT COUNT(*) FROM vote WHERE user_id = ?  
AND matchup_id = ?");
```

```

$voteCheck->execute([$userId, $matchupId]);
$hasVoted = $voteCheck->fetchColumn();

if ($hasVoted) {
    die("You've already voted in this matchup.");
}

// Insert the vote
$insertVote = $db->prepare("
    INSERT INTO vote (user_id, matchup_id, entry_id, cast_at) VALUES (?, ?,
    ?, NOW())");
$insertVote->execute([$userId, $matchupId, $entryId]);

```

Implements rules that the database does not enforce on its own (e.g., early rejection before hitting DB constraints).

5.2.2.8 Ensuring Matchup Status Before Voting

This constraint protects matchups from premature or late voting.

```

if ($now < strtotime($matchup['opens_at']) || $now >
    strtotime($matchup['closes_at'])) {
    die("This matchup is not currently open for voting.");
}

```

Prevents premature or late voting and ensures voting windows align with tournament configuration.

5.2.2.9 Handling File Uploads for Profiles (edit_profile.php)

Example of processing and storing user-uploaded images:

```

if (!empty($_FILES['profile_picture']['name'])) {
    $filename = uniqid() . "_" .
    basename($_FILES["profile_picture"]["name"]);
    $target = "uploads/" . $filename;

    if (move_uploaded_file($_FILES["profile_picture"]["tmp_name"], $target))
    {
        $stmt = $db->prepare("UPDATE users SET profile_picture=? WHERE
        user_id=?");
        $stmt->execute([$filename, $user_id]);
    }
}

```

Processes image uploads, stores files on the server, and updates the user record with the image path.

5.2.2.10 AJAX Constraint Enforcement (check_displayname.php)

Real-time validation of display name availability:

```

$stmt = $db->prepare("SELECT COUNT(*) FROM users WHERE displayname=?");
$stmt->execute([$displayname]);

```

```
echo $stmt->fetchColumn();
```

Supports real-time validation on the client side to prevent duplicate display names before form submission.

5.2.2.11 Round Progression Polling ([check_round_change.php](#))

JavaScript on the client polls this PHP file periodically, and PHP returns the current tournament status. When the status changes, the GUI automatically updates the round display.

```
$query = $db->prepare("
    SELECT tourneystatus
    FROM tournament
    WHERE tournament_id = ?
");
$query->execute([$tournament_id]);
echo $query->fetchColumn();
```

This allows the front-end to detect status changes without page reloads.

5.2.2.12 Authentication Middleware

To manage data flow and security, we implemented a reusable authentication layer in `auth.php`. Functions like `ensure_logged_in()` act as middleware, intercepting HTTP requests before they reach sensitive logic. If a user is not authenticated, the application layer halts execution and redirects them to the login page, preventing unauthorized access to database operations.

```
function is_guest() {
    return (isset($_SESSION["role"]) && $_SESSION["role"] === "guest");
}

function ensure_logged_in() {
    // If not a real logged-in user, show the custom "denied" page
    if (!isset($_SESSION["user_id"]) || is_guest()) {
        // Optional: pass where they came from (e.g., stats) as a query param
        header("Location: guest_denied.php?from=statistics");
        exit;
    }
}
```

5.2.2.13 Server-Side API for Real-Time Check on Username

To support the GUI's live validation features, the application layer exposes Web API endpoints. These PHP scripts accept asynchronous requests from the client (via AJAX) and query the database without reloading the page.

For example, `check_displayname.php` accepts a `username` parameter, queries the `users` table to check for duplicates, and returns a simple text response ("taken" or "available"). This allows the application to enforce unique constraints before the user submits the form.

From the URL:
`https://artemis.cs.csub.edu/~fantasytourney/Final_project/check_displayname.php?displayname=`

```
$db = get_pdo_connection();
$displayname = trim($_GET['displayname']);
$user_id = isset($_GET['user_id']) ? intval($_GET['user_id']) : 0;

// Query to check if name exists for another user
$stmt = $db->prepare("
    SELECT user_id
    FROM users
    WHERE LOWER(displayname) = LOWER(?)
    AND user_id != ?
");
$stmt->execute([$displayname, $user_id]);

echo ($stmt->rowCount() > 0) ? "taken" : "available";
```

5.2.3 - Front-End Layer Programming

The front-end layer is responsible for the user interface and client-side logic. Built using HTML, CSS, and Bootstrap, this layer ensures the application is responsive and interactive. We used JavaScript that runs in the user's browser, handling client-side validation, AJAX polling, real-time validation, and dynamic interactive elements. Below are representative excerpts of the code used to support client-side behavior that connects to the PHP application layer.

5.2.3.1 HTML Structure for Round Voting (`round8.php`)

This HTML snippet shows how matchups are displayed and how voting buttons connect to PHP endpoints:

```
<div class="matchup">
  <div class="entry">
    
    <p><?=$matchup['entryA_name'] ?></p>
    <a href="cast_vote.php?matchup=<?=$matchup['matchup_id'] ?>&entry=<?=$matchup['entryA_id'] ?>"
      class="btn btn-primary">Vote</a>
  </div>

  <div class="entry">
    
```

```

    <p><?= $matchup['entryB_name'] ?></p>
    <a href="cast_vote.php?matchup=<?= $matchup['matchup_id'] ?>&entry=<?=
    $matchup['entryB_id'] ?>"
        class="btn btn-primary">Vote</a>
    </div>
</div>

```

This defines how entries and voting buttons appear on the bracket screen. Buttons send GET requests to `cast_vote.php` (the application layer).

5.2.3.2 CSS Styling for Brackets (`round8.css`)

This file controls the visual layout of the matchups, ensures uniform images and spacing, and applies the project's color palette.

```

.entry-img {
    width: 120px;
    height: 120px;
    object-fit: cover;
    border-radius: 10px;
}

.matchup {
    display: flex;
    justify-content: space-around;
    margin-bottom: 20px;
}

.btn-primary {
    margin-top: 10px;
    background-color: #4A90E2;
    border: none;
}

```

The most complex visual component is the Tournament Bracket. We created a custom CSS structure (`round8.css`) that uses Flexbox to align matchups into rounds (Round of 16 → Quarter Finals → Semi-Finals → Final). Pseudo-elements (`:before`, `:after`) are used to draw the connecting lines between matches dynamically.

```

.tournament-headers {
    flex-grow: 1;
    display: flex;
    flex-direction: row;
    justify-content: space-around;
    border-bottom: 1px solid #ccc;
}

.tournament-brackets {
    display: flex;
}

```

```
flex-direction: row;
list-style-type: none;
background: #fdfdfd;
margin-bottom: 50px;
}

.bracket {
padding-left: 0;
display: flex;
margin: 0;
padding: 30px 0;
flex-grow: 1;
flex-direction: column;
justify-content: space-around;
list-style-type: none;
border-right: 1px dashed #ccc;
flex: 1;
}

.team-item {
background-color: #f4f4f4;
padding: 0.5rem;
display: block;
margin: 0.5rem 10px;
position: relative;
line-height: 2;
text-align: center;
}

.team-item:after {
content: "";
border-color: #4f7a38;
border-width: 2px;
position: absolute;
display: block;
width: 10px;
right: -11px;
}

.team-item:nth-of-type(odd):after {
border-right-style: solid;
border-top-style: solid;
height: 100%;
top: 50%;
}

.team-item:nth-of-type(even):after {
border-right-style: solid;
border-bottom-style: solid;
```

```

height: 100%;
top: -50%;
}

.team-item:before {
content: "";
border-top: 2px solid #4f7a38;
position: absolute;
height: 2px;
width: 10px;
left: -10px;
top: 50%;
}

```

5.2.3.3 AJAX: Checking Display Name Availability

To improve the user experience, we implemented AJAX requests using the JavaScript `fetch()` API. This allows the application to validate data and update the interface in real-time without reloading the page

On the Registration page (`register.php`, `edit_profile.php`), an event listener detects keystrokes in the "Display Name" field. It sends an asynchronous request to the application layer to check availability. This provides immediate visual feedback (green tickmark/ red X) to the user.

```

<script>
function checkName() {
  let name = document.getElementById("displayname").value;

  fetch("check_displayname.php?name=" + name)
    .then(response => response.text())
    .then(count => {
      if (count > 0) {
        document.getElementById("nameStatus").innerHTML =
          "Display name already taken.";
      } else {
        document.getElementById("nameStatus").innerHTML =
          "Display name available.";
      }
    })
  });
}
</script>

```

It sends an AJAX request to a PHP endpoint without reloading the page. Provides instant feedback to the user before submitting a form.

5.2.3.4 AJAX Round Status Polling

Uses JavaScript polling (setInterval) every 5 seconds. Calls a PHP endpoint that returns the current round/tournament status. If the status changes (round completed), the GUI refreshes automatically. Enables real-time bracket updates without user interaction.

```
<script>
setInterval(() => {
  fetch("check_round_change.php?tournament_id=<?= $tournament_id ?>")
    .then(res => res.text())
    .then(status => {
      if (status !== currentStatus) {
        location.reload(); // refresh UI automatically
      }
    });
}, 5000);
</script>
```

5.2.3.5 Client-Side Form Validation (register.php)

Ensures basic input validation before submitting to the server.

```
<script>
function validateForm() {
  const email = document.getElementById("email").value;
  if (!email.includes("@")) {
    alert("Please enter a valid email.");
    return false;
  }
  return true;
}
</script>
```

Reduces invalid server requests.

5.2.3.6 HTML Form Example (Profile Editing)

Demonstrates how forms send data from the user to PHP scripts.

```
<form action="update_profile.php" method="POST" enctype="multipart/form-data">
  <label>Display Name:</label>
  <input type="text" id="displayname" name="displayname" onkeyup="checkName()">

  <p id="nameStatus" class="text-muted"></p>

  <label>Profile Picture:</label>
  <input type="file" name="profile_picture">

  <button type="submit" class="btn btn-primary">Save Changes</button>
```

```
</form>
```

Links directly to AJAX validation. Supports file uploads to the server.

5.2.3.7 Front-End Table Rendering (statistics.php)

Dynamically generates HTML from PHP data. Displays analytics to players and admins.

```
<table class=" table table-striped">
  <tr><th>Entry</th><th>Total Votes</th></tr>
  <?php foreach ($top5 as $row): ?>
    <tr>
      <td><?= $row['entry_name'] ?></td>
      <td><?= $row['total_votes_received'] ?></td>
    </tr>
  <?php endforeach; ?>
</table>
```

5.2.3.8 Report Generation

For the reporting requirement, we utilized the browser's native capabilities. The Statistics Page allows users to export data as a PDF. We attached a simple JavaScript trigger to a button that invokes the print dialog, which gives out a clean, professional report.

```
<button onclick="window.print()" class="btn btn-secondary btn-sm ">
  Save as PDF
</button>
```

5.2.3.9 Interactive UI Elements (Tooltips)

We implemented dynamic tooltips using Bootstrap 5. On the Tournament Bracket page (round8.php), we display the tournament creator's profile picture and username. Hovering over the username triggers a JavaScript-initialized tooltip that displays the creator's full bio. This was achieved by adding data-bs-toggle="tooltip" attributes to the HTML elements and initializing them via a global JavaScript function at the bottom of the page layout.

```
const tooltipTriggerList =
document.querySelectorAll('[data-bs-toggle="tooltip"]')
const tooltipList = [...tooltipTriggerList].map(tooltipTriggerEl => new
bootstrap.Tooltip(tooltipTriggerEl))
```